



Parallel k -Core Decomposition: Theory and Practice

YOUZHE LIU, University of California, Riverside, USA

XIAOJUN DONG, University of California, Riverside, USA

YAN GU, University of California, Riverside, USA

YIHAN SUN, University of California, Riverside, USA

This paper proposes efficient solutions for k -core decomposition with high parallelism. The problem of k -core decomposition is fundamental in graph analysis and has applications across various domains. However, existing algorithms face significant challenges in achieving work-efficiency in theory and/or high parallelism in practice, and suffer from various performance bottlenecks.

We present a simple, work-efficient parallel framework for k -core decomposition that is easy to implement and adaptable to various strategies for improving work-efficiency. We introduce two techniques to enhance parallelism: a sampling scheme to reduce contention on high-degree vertices, and vertical granularity control (VGC) to mitigate scheduling overhead for low-degree vertices. Furthermore, we design a hierarchical bucket structure to optimize performance for graphs with high coreness values.

We evaluate our algorithm on a diverse set of real-world and synthetic graphs. Compared to state-of-the-art parallel algorithms, including ParK, PKC, and Julienne, our approach demonstrates superior performance on 23 out of 25 graphs when tested on a 96-core machine. Our algorithm shows speedups of up to 315 $\times$ over ParK, 33.4 $\times$ over PKC, and 52.5 $\times$ over Julienne.

CCS Concepts: • **Theory of computation** → **Graph algorithms analysis; Parallel algorithms; Shared memory algorithms.**

Additional Key Words and Phrases: Parallel Algorithms, Graph Algorithms, k -Core Decomposition, Work-Efficiency

ACM Reference Format:

Youzhe Liu, Xiaojun Dong, Yan Gu, and Yihan Sun. 2025. Parallel k -Core Decomposition: Theory and Practice. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 195 (June 2025), 27 pages. <https://doi.org/10.1145/3725332>

1 Introduction

Analyzing real-world graphs is crucial in a wide range of applications. One of the most widely-used techniques for identifying densely connected regions in a network is the k -core decomposition [56, 67]. This problem about subgraph structure detection finds applications in various fields, including social network analysis [12, 31, 41, 43, 55, 81], risk assessment [13, 30, 43, 55, 60, 83], bioinformatics [15, 26, 43, 55, 78], and system robustness analysis [13, 43, 55, 72, 80].

Given a graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges, the k -core of G is the maximal subgraph $G' = (V', E')$ of G in which every vertex has degree at least k (see an illustration in Fig. 1). The k -core decomposition of a graph G identifies a sequence of non-empty subgraphs

Authors' Contact Information: Youzhe Liu, yliu908@ucr.edu, University of California, Riverside, Riverside, CA, USA; Xiaojun Dong, xdong038@ucr.edu, University of California, Riverside, Riverside, CA, USA; Yan Gu, ygu@cs.ucr.edu, University of California, Riverside, Riverside, CA, USA; Yihan Sun, yihans@cs.ucr.edu, University of California, Riverside, Riverside, CA, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/6-ART195
<https://doi.org/10.1145/3725332>

$G_0, G_1, \dots, G_{k_{\max}}$ for all possible k values, where G_i is the i -core of G . The **coreness** of a vertex, denoted as $\kappa[v]$, is the maximum value of k such that v is in G_k . The coreness of a graph, denoted as $k_{\max}$, is the maximum coreness among all vertices. The output of the k -core decomposition is the coreness for each vertex, which can be used to reconstruct any G_i .

Consider the ever-growing size of today's real-world graphs, which can reach terabyte scale, it is essential to consider parallelism in the k -core computation. In this paper, we focus on the high-performance algorithm to compute exact k -core decomposition with high parallelism. While k -core is simple in the sequential setting, and various efficient solutions exist [9, 67], parallel k -core is a notoriously hard problem. Starting from $k = 0$, the sequential algorithm keeps removing vertices with degree no more than k and update their neighbors' degrees, until all nodes have a degree at least $k + 1$, obtaining the $(k + 1)$ -core. Then the algorithm will increment k and repeat this process. An example of this process, usually referred to as the "peeling" process, is given in Fig. 1. This sequential approach requires $O(n + m)$ number of operations (aka. work or time complexity). However, parallelizing this sequential method is highly non-trivial. Despite k -core is studied in many papers and implemented in many libraries [2, 14, 17, 18, 27, 33, 34, 38, 39, 42, 46, 50, 57, 59, 70, 73, 77, 82, 84, 85], many challenges remain, both in theory and in practice, to achieve a parallel k -core algorithm that is simple, efficient, and scalable on various types of graphs. For instance, in Fig. 2, we show that on a 96-core machine, each state-of-the-art parallel k -core solution can be slower than a sequential implementation on certain graphs, and the "worst cases" vary significantly between algorithms. This highlights the intricacies of optimizing the k -core algorithm in the parallel setting.

In this paper, we propose a parallel k -core algorithm with a series of novel algorithmic techniques. Our algorithm is efficient, practical, and achieves high performance across various graph types. In the following, we briefly overview the challenges in existing work, and our solutions to overcome them.

Theoretical challenges and our contributions. The probably most surprising challenge we have observed is that, we are unaware of any analysis on work-efficiency¹ for the existing parallel implementations. Most of existing parallel k -core decomposition algorithms have $O(m + k_{\max}n)$ work, rendering a significant overhead caused by parallelism. The only known work-efficient parallel k -core algorithm is from Julienne [18]. Unfortunately, this algorithm is mainly of theoretical interest. During the entire peeling process, it maintains a *bucketing structure*, which maps each possible degree d to all vertices with degree d . However, maintaining the full mapping incurs performance overheads and coding complexity in practice. Hence, their implementation [18] did not adopt the bucketing structure as described and used a simplified alternative, which only maintains at most b buckets at any time. It remains unknown if their simplified implementation is theoretically efficient.

In this paper, we present a surprisingly simple parallel k -core algorithm, shown in Alg. 1, with $O(n + m)$ work. Our framework does not need the complicated bucketing structure to achieve work-efficiency, making it simple to implement. Our analysis indicates that the simplified implementation of Julienne is also work-efficient, and some other existing algorithms with $O(m + k_{\max}n)$ work can be made work-efficient with minor modifications.

Practical challenges and our contributions. While our analysis indicates that achieving work-efficiency in k -core algorithms is not hard, translating this into a fast, practical implementation is challenging due to the difficulty of achieving high parallelism. Existing implementations perform the peeling process in either an online or an offline manner. When peeling vertices with degree k , the offline approach, implemented in Julienne [18], chooses to collect all vertices that require degree

¹The *work* of a parallel algorithm is its time complexity running on one core. A parallel algorithm is *work-efficient* if its work is the same as the best sequential time complexity.

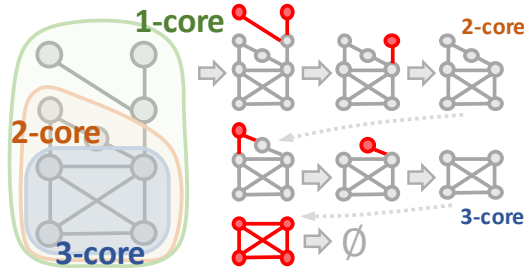


Fig. 1. An example of k -core decomposition with $k_{\max} = 3$. Vertices and edges peeled in each subround are marked as red.

decrement in a batch, counts the number of occurrences for each of them, and applies all of them in parallel. The benefit of the algorithm is race-freedom. However, since this approach is fully synchronous, meaning that the degree decrement for each batch (which we call a *subround* in this paper) must fully finish before the next batch starts, causing high overhead to synchronize threads between subrounds. One adversarial example is a $\sqrt{n} \times \sqrt{n}$ grid (the *GRID* graph in Fig. 2), that incurs $O(\sqrt{n})$ subrounds. An illustration of this process is given in Fig. 3(a). On this graph, *Julienne* is slower than a sequential implementation due to high scheduling overhead.

Many other parallel k -core implementations, such as PKC [38] and ParK [17], use the *online* approach. When peeling a vertex v , the degrees of its neighbors are directly decremented via atomic operations. The online algorithm follows a simple framework, and certain optimizations can also be used to reduce the synchronization cost [38]. However, for high-degree vertices, a large number of concurrent decrements can cause heavy contention, which degrades performance. As a result, PKC and ParK are slower than sequential algorithms on dense networks (e.g., *TW* and *SD* in Fig. 2).

To resolve these challenges, we propose several novel techniques. We adopt the online framework with two additional techniques, given in Sec. 4. The first is a *sampling* scheme to reduce concurrent decrements on high-degree vertices. For a vertex v with a large degree, we select a subset of its neighbors, and only the removal of these neighbors decrements v 's degree. The key challenge of sampling is to accurately estimate the true degree of each vertex v , and to identify v when it is ready to be peeled. Our analysis ensures the sampling algorithm provides correct estimates with high probability. On dense graphs, sampling improves the performance of our code by up to $4.31\times$.

Our second technique is to reduce the high synchronization cost between peeling subrounds. By using the online framework, our algorithm allows asynchronous degree decrements. When we peel a low-degree vertex v , the actual computation to process all its neighbors is minimal, and the scheduling overhead to create and synchronize this thread may dominate the cost. To address this, we propose a *local search* algorithm to reduce the number of subrounds and hide scheduling overhead. This technique improves the performance of our code by up to $31.2\times$.

Finally, we propose a new hierarchical bucketing structure (HBS) to further improve performance in Sec. 5. Instead of using no bucket structure (or equivalently, using a single bucket) as in our proposed framework, or a fixed number of buckets as in *Julienne*, HBS efficiently manages vertices in each round. On average (geometric mean across all graphs), our new design improves the performance by $1.6\times$ compared to the plain version (no-bucket structure), and is $1.3\times$ faster than using 16 buckets.

With our new framework and techniques, our algorithm achieves high performance in experiments. We compare our implementation against three state-of-the-art parallel implementations: *Julienne*, ParK and PKC, testing a diverse set of graphs, including social networks, web graphs, road networks, k -NN graphs, and other graphs including synthetic graphs and simulation graphs. Due to different designs, each baseline has lower performance than a sequential implementation on certain

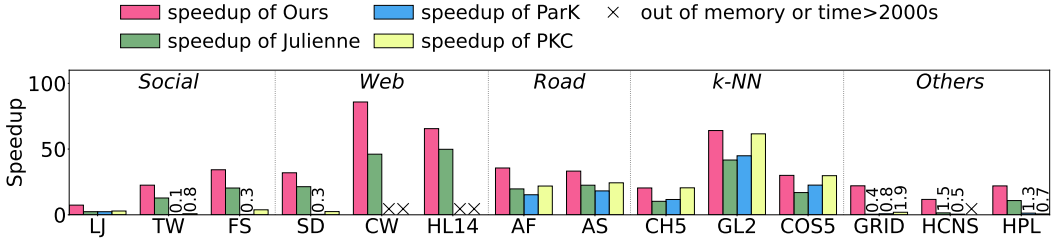


Fig. 2. Speedup of Park [17], PKC [38], Julienne [18, 19], and our algorithm, relative to the best sequential time (BZ [9] or ours) on 14 graphs.

graphs. Our algorithm consistently performs well on all types of graphs, outperforming the best sequential algorithm by 7.3–84 $\times$. On dense graphs, our algorithm is faster than all baselines, with 1.14–3.17 $\times$ faster than Julienne, 2.6–315 $\times$ faster than Park, and 1.5–28 $\times$ faster than PKC. On most sparse graphs, our algorithm outperforms the baselines, except for two cases where it is within 12% of the best baseline. On sparse graphs, our algorithm is up to 53 $\times$ faster than Julienne, 28 $\times$ faster than Park, and 11 $\times$ faster than PKC. We also carefully evaluated the performance gain from the three new techniques in experiments.

We release our code with the link given in [51]. Due to the page limit, the full version of this paper is given in [52].

2 Preliminaries

Notations. Given an undirected graph $G = (V, E)$, the k -core of a graph G , noted as G_k , is the largest subgraph of G where every vertex has degree at least k . The **coreness** of a vertex, noted as $\kappa(v)$, is the maximum value of k such that v is in G_k . The coreness of a graph, noted as $k_{\max}$, is the maximum coreness among all vertices. The **k -core decomposition** of G identifies the sequence of subgraphs $G_0, G_1, \dots, G_{k_{\max}}$. With clear context, we use k -core to refer to k -core decomposition. Our algorithm computes the coreness for each vertex $v \in V$, which can be used to recover any G_i .

Most k -core algorithms are based on the *peeling* process, where vertices with the smallest degrees are removed, and their neighbors' degrees are updated accordingly. We use $d(v)$ to represent the degree of a vertex in the original graph. To distinguish from the original degree, we use **induced degree** to denote the degree of v during the peeling process, which may be decremented by the removal of its neighbors, and denote it as $\tilde{d}[v]$ for vertex v .

We say $O(f(n))$ with high probability (*whp*) in n to indicate $O(cf(n))$ with probability at least $1 - n^{-c}$ for any $c \geq 1$. We say an event happens with high probability (*whp*) in n to indicate that probability is at least $1 - n^{-c}$ for any $c \geq 1$ as a parameter. When clear from context we drop the “in n ”.

Parallel Computational Model. We use the work-span model in the classic multithreaded model with binary fork-join [10, 11]. We assume a set of threads that share a common memory. A process can fork two child software threads to work in parallel. When both children complete, the parent process continues. The **work** of an algorithm is the total number of instructions. An algorithm is **work-efficient** if its work is asymptotically the same as the best sequential algorithm. The **span** of an algorithm is the length of the longest dependence chain among operations. We can execute a computation with W work and S span with using a randomized work-stealing scheduler [6, 11, 35] in time $W/P + O(S)$ *whp* in W .

A major goal in our work is to reduce the synchronization cost in the k -core algorithm. Unfortunately, this cost is not well-reflected by the classic work-span model. More precisely, while each fork/join operation is counted as a constant cost in theory, such a constant is usually large due to

Notations for the input graph:

$G = (V, E)$	Graph G with vertex set V and edge set E
n, m	$n = V , m = E $
$\kappa(v)$	The coreness of vertex v
$k_{\max}$	The maximum coreness value in the graph
$N(v)$	The set of neighbors of vertex v
$d(v)$	The degree of vertex v

Notations used in the algorithm:

$\mathcal{A}$	The set of active vertices (not peeled yet)
$\mathcal{F}$	The set of frontier vertices
$\tilde{d}[v]$	The induced degree of v during the algorithm
k	The current peeling round
Notations used in sampling are defined in Alg. 5	

Table 1. Notation used in this paper.

the scheduling scheme. To formally analyze the cost, we borrow the concept of **burdened span** from Cilkview [37], a profiling tool for multicore programs. Burdened span charge a cost of ω for a fork/join operation to reflect scheduling overhead, and a unit cost as usual for other operations. We use the default parameter $\omega = 15,000$ from the original Cilkview paper. We will show that our proposed technique improves the burdened span compared to existing solutions. We also use Cilkview [37] to measure the exact burdened span on real-world graphs in our experiments.

We use atomic operations `atomic_inc( $p$ )` and `atomic_dec( $p$ )`, which atomically reads the memory location pointed to by p , and increment (or decrement) the integer value stored at p by 1. The function returns the original value stored at p . We assume constant work for each atomic operation in theoretical analysis. We note that, however, the cost for atomic operations can usually be large in practice, especially when many threads are concurrently updating the same memory location. One effort in our paper is to avoid such high contention by reducing the number of atomic updates to the same memory location. To capture this, we define **contention** [1] experienced by an operation as the maximum number of operations that may concurrently modify the same memory location. The contention of an algorithm is the highest contention among all operations in the algorithm. In Sec. 4.1, we will show that our sampling technique effectively reduces the contention.

Parallel Primitives. The `PACK` function is a widely-used parallel primitive in this paper. Given an array A of elements and a predicate function f , the `pack` function returns all elements in A that satisfy f in a new array. This function takes $O(|A|)$ work and can be highly parallelized. Julianne uses the `HISTOGRAM` function, which takes an array A of keys, and count the occurrences of each key in A . It can be computed efficiently using parallel semisort [25, 36].

Parallel Hash Bag. In this paper, we use a data structure *parallel hash bag* [24, 74]. A parallel hash bag supports concurrent insertion and resizing, and extracting all elements into a consecutive array. More precisely, a parallel hash bag supports two operations:

- `BAGINSERT( $v$ )`: (concurrently) add the element v into the bag.
- `BAGEXTRACTALL()`: extract all elements in the bag into an array.

Hashing is a widely-used technique for accelerating parallel data access [4, 21, 22, 71]. Similar to a hash table [71], a hash bag maintains a set of elements by hashing every element into certain index, and resolve conflicts by linear probing. A hash bag is initialized as an array with $O(n)$ slots, where n is the maximum possible number of elements appearing in the bag at the same time. The array is conceptually divided into chunks of sizes $\lambda, 2\lambda, 4\lambda, \dots$ ($\lambda = 2^8$ in implementation). At the beginning, elements are inserted into the chunk of size λ . Once the current chunk reaches a desired load factor, we move onto the next chunk of size 2λ , and so on so forth. The `BAGEXTRACTALL()`

function only needs to consider the prefix chunks that have been used instead of the entire array. Therefore, the complexity of `BAGEXTRACTALL()` is $O(\lambda + t)$, where t is the number of elements in the hash bag. In this paper, we use parallel hash bag to maintain the frontiers and some vertex sets in our algorithm.

Bucketing Structure for k -core Algorithms. The bucketing structure organizes data with integer keys and supports efficient updates and finding the element with the smallest key. A parallel bucketing structure has been proposed by [18] to support the k -core algorithm in Julienne. The design of the bucketing structure has been widely studied [18, 19, 47, 68, 69] and has many applications. In Sec. 5, we present more details about the existing bucketing structure and our new design, which not only enhances the performance of the k -core algorithm but is also of independent interest.

3 Algorithm Framework and Design

In this section, we present our algorithmic framework for k -core with theoretical analysis. Many existing implementations fit into this framework. However, as mentioned, we are unaware of any analysis to prove their work-efficiency. For example, both ParK [17] and PKC [38] proved $O(k_{\max}n + m)$ time complexity for their algorithms, where $k_{\max}$ can be $O(\sqrt{m})$ in the worst case. Julienne [18] introduced a k -core algorithm with $O(m+n)$ work. However, this algorithm is mostly of theoretical interest. Due to complicated algorithmic details that may cause a performance overhead in practice, their open-source code implemented a simpler (and more practical) version instead of the algorithm described in their paper. It is therefore unclear what the cost is for their implementation. This motivates us to consider *whether there exists a simple and work-efficient parallel k -core solution*. Interestingly, we observed that a simple algorithmic framework can essentially give $O(m + n)$ work. Our results show that the implementation in Julienne is indeed theoretically efficient, as well as a simple variant of ParK or PKC. In fact, while many existing algorithms roughly follow the high-level idea of the framework, we believe that the formalization of the framework helps enable simple and general analysis for work-efficiency, reveal the advantages/disadvantages of existing approaches, and inspires the main techniques in this paper. In the following, we show the framework in Alg. 1, describe and analyze it in Sec. 3.1, and finally discuss different strategies for the peeling process in Sec. 3.2.

3.1 Framework

We present our algorithm framework in Alg. 1. Given a graph $G = (V, E)$, the algorithm returns the coreness $\kappa[v]$ of each $v \in V$.

As with the existing work, our algorithm is also *frontier-based*, where vertices with the same degree are peeled in parallel, organized as a *frontier*, denoted as $\mathcal{F}$. In addition, our algorithm maintains a set of **active vertices** $\mathcal{A}$, which are the vertices that have not been peeled. We call $\mathcal{A}$ the **active set**. In the peeling process, we use $\tilde{d}[v]$ to track the induced degree of v . We first initialize $\tilde{d}[v]$ as the degree of v in the input graph, and $\mathcal{A}$ as the entire vertex set V . We use k to denote the current peeling round, starting with $k = 0$. In round k , we first initialize the frontier $\mathcal{F}$ by extracting the vertices from $\mathcal{A}$ with induced degree k (line 5). The coreness of all vertices in the frontier can be set to k . We then peel all vertices in $\mathcal{F}$ (line 1). For each vertex $v \in \mathcal{F}$, peeling it will decrement the induced degree for each of its neighbors by 1. When the induced degree of any vertex u hits k , u is added to a set $\mathcal{F}_{\text{next}}$, which becomes the next frontier. We repeat this process until the frontier becomes empty. We call each iteration on line 6 a **subround**, which deals with one frontier, and call each iteration on line 4 a **round**, which can contain multiple subrounds with

Algorithm 1: Work-efficient parallel k -core framework**Input:** Graph $G = (V, E)$ **Output:** The coreness for each vertex

```

1   $\tilde{d}[\cdot] \leftarrow d(\cdot)$  // Initialize the induced degree set of all vertices
2   $\mathcal{A} \leftarrow V$  // Active set: vertices that have not been peeled
3   $k \leftarrow 0$ 
4  while  $\mathcal{A} \neq \emptyset$  do
5       $\mathcal{F} \leftarrow \{v \mid v \in \mathcal{A}, \tilde{d}[v] = k\}$  // The initial frontier in round  $k$ 
6      while  $\mathcal{F} \neq \emptyset$  do
7          foreach  $v \in \mathcal{F}$  do  $\kappa[v] \leftarrow k$  // Sets coreness to  $k$ 
8           $\mathcal{F} \leftarrow \text{PEEL}(\mathcal{F}, k)$ 
9       $\mathcal{A} \leftarrow \{v \mid v \in \mathcal{A}, \tilde{d}[v] > k\}$  // Refines the active set
10      $k \leftarrow k + 1$ 
11 return  $\kappa[\cdot]$ 

/* A sequential version. Parallel versions are discussed in Sec. 3.2. */
12 Function  $\text{PEEL}(\mathcal{F}, k)$ 
13     Initialize  $\mathcal{F}_{\text{next}} \leftarrow \emptyset$  // Buffers the next frontier
14     foreach  $v \in \mathcal{F}$  do
15         foreach  $u \in N(v)$  do
16              $\tilde{d}[u] \leftarrow \tilde{d}[u] - 1$ 
17             if  $\tilde{d}[u] = k$  then Add  $u$  to  $\mathcal{F}_{\text{next}}$ 
18     return  $\mathcal{F}_{\text{next}}$  // Returns the next frontier

```

the same coreness value k . After each round, we update the active set $\mathcal{A}$ (line 9) by only keeping vertices with induced degrees larger than k .

The cost of the algorithm relies on three parts: 1) line 5 that extracts the initial frontier of a round, 2) the PEEL function that processes all neighbors of the frontier, and 3) line 9 that refines the active set $\mathcal{A}$. Next, we will show that, with proper implementations of the three parts, the framework in Alg. 1 has $O(n + m)$ work.

THEOREM 3.1. Assuming $O(|\mathcal{F}| + \sum_{v \in \mathcal{F}} d(v))$ work for the $\text{PEEL}(\mathcal{F}, \cdot)$ function on line 8, and $O(|\mathcal{A}|)$ work for the functions on lines 5 and 9, where $\mathcal{A}$ is the input active set), the total work of Alg. 1 is $O(n + m)$.

Proof. The main work of the algorithm comes from two parts: the $\text{PEEL}()$ function, and maintaining the sets $\mathcal{F}$ and $\mathcal{A}$ on lines 5 and 9.

For the PEEL function, note that each vertex will appear in exactly one frontier. As we assumed in the theorem, the work of PEEL is proportional to the total number of neighbors of vertices in the frontier. Therefore the total work is $\sum_{v \in V} (1 + d(v)) = O(n + m)$.

We now analyze the two functions that generates $\mathcal{F}$ on line 5 and $\mathcal{A}$ on line 9. The theorem assumes that the cost for both of them is proportional to the input size $|\mathcal{A}|$, and thus they have the same asymptotic cost. Let $\mathcal{A}_i$ be the active set of round i , which contains all vertices with coreness at least i . The total cost is:

$$\begin{aligned}
 \sum_{i=0}^{k_{\max}} |\mathcal{A}_i| &= \sum_{i=0}^{k_{\max}} |\{v \mid v \in V, \kappa[v] \geq i\}| \\
 &= \sum_{v \in V} (1 + \kappa[v]) \leq \sum_{v \in V} 1 + \sum_{v \in V} d(v) = O(n + m)
 \end{aligned}$$

The third step uses the fact that $\kappa(v) \leq d(v)$ for all $v \in V$. □

Algorithm 2: Offline peeling process

```

1 Function PEEL( $\mathcal{F}, k$ )
2    $L \leftarrow$  the list of vertices  $u$ , s.t.  $(u, v) \in E, v \in \mathcal{F}$ ; duplicates are kept.
3    $H \leftarrow$  HISTOGRAM( $L$ ) // Count the frequency for each  $u$  in  $L$ 
4   parallel_for ( $u, f_u$ )  $\in H$  do // for each  $u$  with frequency  $f_u$ 
5     if  $\tilde{d}[u] > k$  then  $\tilde{d}[u] \leftarrow \tilde{d}[u] - f_u$ 
6    $\mathcal{F}_{\text{next}} \leftarrow \{u \in L, \tilde{d}[u] \text{ dropped to } k \text{ or lower by line 5} \}$ 
7   return  $\mathcal{F}_{\text{next}}$ 

```

The key in the analysis is to bound to total size of all active sets. Although it does not seem too complicated, to the best of our knowledge, we are unaware of existing work with this result.

Note that the two functions on line 5 and line 9 can be implemented in $O(|\mathcal{A}|)$ work by the standard parallel PACK function introduced in Sec. 2. Hence, the key component in this framework is an efficient PEEL function. In the following, we will introduce two main peeling approaches for in existing implementations, how they can be analyzed using our framework, and their advantages and drawbacks to process certain types of graphs.

3.2 Parallel Peeling Process in Existing Work

Under the proposed framework, multiple strategies for solving the PEEL function on line 8 in parallel can be applied. We first review the two most relevant strategies in the literature.

The Offline Strategy in Julienne. Julienne [18] employs a batch-synchronous strategy for the PEEL function (Alg. 2). In each subround, it gathers all vertices that require degree changes in a list L , which concatenates the neighbor lists for all vertices in $\mathcal{F}$. Each appearance of a vertex u in L means to decrement the induced degree $\tilde{d}[u]$ by 1. Hence, a HISTOGRAM algorithm is used to count the number of appearances for each vertex in the list, which can be performed by a parallel semisort [25, 36] with $O(n)$ work with high probability. The induced degree for each vertex in L will be decremented in a batch accordingly, and the next frontier can be computed as all vertices that have degree drop to k or lower by a parallel pack. We refer to this approach as an *offline* approach. Since each vertex and edge is processed exactly once in the peeling process, the total work for this step is proportional to the total neighborhood size of the vertices in the frontier. Based on Thm. 3.1, the k -core implementation in Julienne has $O(n + m)$ work.

The actual implementation of Julienne is more complicated than our framework. Our new algorithm with offline peeling is much simpler than both their theoretical algorithm in the paper and their implementation. Our result indicates that achieving work-efficiency does not require the complicated bucketing structure. We note that adding a bucketing structure can likely improve the performance in practice, which we discuss in Sec. 5.1.

While Julienne achieves efficient work, the parallelism of Julienne can be bottlenecked the scheduling overhead caused by the offline algorithm. Note that each subround requires to distribute work to all threads and synchronize them at the end. On sparse graphs, this scheduling overhead can dominate the actual computation cost, making Julienne slower than a sequential algorithm in certain cases. In Sec. 4.2 we discuss more details and provide our solution to overcome this challenge.

The Online Strategy in Park and PKC. Both Park [17] and PKC [38] use an asynchronous peeling algorithm (Alg. 3), which removes vertices in the frontier in parallel, and directly decrements the induced degrees of their neighbors using the atomic_dec operation. We refer to this approach as an *online* approach, since when we process a vertex v in the frontier, the induced degrees $\tilde{d}[\cdot]$ of its neighbors are updated immediately. When $\tilde{d}[u]$ is decremented from $k + 1$ to k , u will be

Algorithm 3: Online peeling process

```

1 Function PEEL( $\mathcal{F}, k$ )
2   Initialize  $\mathcal{F}_{\text{next}} \leftarrow \emptyset$  // Buffers the next frontier
3   parallel_foreach  $v \in \mathcal{F}$  do
4     parallel_foreach  $u \in N(v)$  do
5        $\delta \leftarrow \text{atomic\_dec}(\tilde{d}[u])$  // decrement atomically
6       // The last decrement adds u to the next frontier
7       if  $\delta = k + 1$  then Add  $u$  to  $\mathcal{F}_{\text{next}}$ 
8   return  $\mathcal{F}_{\text{next}}$  // Returns the next frontier

```

put in the next frontier by atomically appending it to an array $\mathcal{F}_{\text{next}}$. Assuming constant work for each `atomic_dec` operation and appending each element to $\mathcal{F}_{\text{next}}$, each peeling subround has cost proportional to the total neighborhood size of the vertices in the frontier.

Unfortunately, neither of the two algorithms maintains the active set during the algorithm, and simply use the original vertex set V to generate each frontier. Therefore, the two algorithms both have $O(m + k_{\max}n)$ work. However, introducing the active set in the algorithm will directly lead to work-efficiency.

The computation in the online approach is simpler than the offline approach, and does not require strong synchronization between subrounds. Certain optimizations can be used to alleviate the scheduling overhead (e.g., PKC [38] and our new solution in Sec. 4.2). The major performance bottleneck of ParK and PKC comes from the potential contention caused by the atomic operations. For a high-degree vertex, many of its neighbors can decrement its induced degree concurrently. As we can observe from the results in Tab. 2, ParK and PKC both suffer from poor performance on power-law graphs such as social and web graphs. Even a small fraction of high-degree vertices may result in high contention that degrade the performance. In addition, the data structure $\mathcal{F}_{\text{next}}$ to generate the next frontier may also suffer from contention when many vertices are appended concurrently. In Sec. 4.1, we propose our solution to overcome this challenge.

Span Analysis. To analyze the span of the algorithm, Julianne [18] defined a parameter ρ to represent the number of peeling subrounds, referred to as the *peeling complexity*, and showed that the span of their algorithm is $\tilde{O}(\rho)$ *whp*. For the framework in Alg. 1 with both online and offline peeling algorithms, the same $\tilde{O}(\rho)$ span bound holds (*whp* for the offline version) following the same analysis in Julianne. However, note that each of the ρ subrounds requires a parallel-for loop, indicating a global synchronization among threads. Therefore, the *burdened span* (defined in Sec. 2) is $\tilde{O}(\rho\omega)$. Given ω as a large constant, when ρ is also large, the parallelism in the algorithm can be limited. In Sec. 4.2, we discuss our new techniques that improve the burdened span, thus parallelism.

4 Our New Techniques

In this section, we introduce our new techniques to overcome challenges in existing solutions. Our algorithm is also based on the online algorithm shown in Alg. 3. We use the parallel hash bag introduced in Sec. 2 to maintain the frontiers in the algorithm.

As discussed, there exist two main challenges in existing solutions that limits parallelism on different types of graphs. The first occurs on graphs with high-degree vertices, where the atomic operations in the online algorithm cause high contention. The second occurs on sparse graphs with low-degree vertices, where synchronization between subrounds causes high scheduling overhead. In this section, we propose our solutions to these two challenges, including a *sampling scheme* to

Algorithm 4: Our algorithm framework with sampling

```

1 Input: Graph  $G = (V, E)$ . Output:  $\kappa[\cdot]$ : Coreness of each vertex   Initialize  $\mathcal{A} \leftarrow V$ ;  $k \leftarrow 0$ ;  $\tilde{d}[v] \leftarrow d(v)$ 
   for all  $v \in V$ 
2 parallel_foreach  $v \in V$  do SETSAMPLER( $v, 0$ )                                     // Initialize  $\sigma[v]$ 
3 while  $\mathcal{A} \neq \emptyset$  do
4    $\mathcal{F} \leftarrow \{v \mid v \in \mathcal{A}, \tilde{d}[v] = k\}$ 
5   parallel_foreach  $v \in V$  :  $v$  is in sample mode do
6     if VALIDATE( $v, k$ ) = false then RESAMPLE( $v, k, \mathcal{F}$ )
7   while  $\mathcal{F} \neq \emptyset$  do
8     parallel_foreach  $v \in \mathcal{F}$  do  $\kappa[v] \leftarrow k$ 
9      $\langle \mathcal{F}, C \rangle \leftarrow \text{PEEL}(\mathcal{F}, k)$                                      //  $C$ : vertices that require to reset samplers
10    parallel_foreach  $v \in C$  do RESAMPLE( $v, k, \mathcal{F}$ )
11     $\mathcal{A} \leftarrow \{v \mid v \in \mathcal{A}, \tilde{d}[v] > k\}$ 
12     $k \leftarrow k + 1$ 
13 return  $\kappa[\cdot]$ 

```

reduce high contention on concurrent atomic operations, and a *vertical granularity control* approach that uses a local search to reduce the number of subrounds.

4.1 Reducing Contention Using Sampling

As mentioned, one challenge in the online peeling algorithm is the high contention in decrementing the induced degree of each vertex concurrently, especially for high-degree vertices. To overcome this challenge, we propose a sampling scheme. In particular, when the degree of a vertex v is over a certain threshold, we will turn it on *sample mode*. When decrementing the induced degree of a vertex in the sample mode, instead of atomically decrementing $\tilde{d}[v]$, we will take a sample for it with a certain probability, which we call the *sample rate*. Based on the number of samples and the sample rate, we can estimate the expectation of the current induced degree. When the expectation is far above k , it is unlikely that $\tilde{d}[v]$ drops to k , and thus we do not need to know its exact induced degree in the current round. In other words, for a high-degree vertex v , we do not update $\tilde{d}[v]$ explicitly before we collect sufficient samples, thus avoiding updating $\tilde{d}[v]$ frequently.

To do this, we maintain a structure *sampler* for each vertex v (defined in Alg. 5), recording whether v is in the sample mode, its sample rate, and the number of samples taken so far. Based on the sampler of a vertex v , we can estimate the expectation of the true induced degree of v , as well as the probability that the true induced degree is lower than k . If the true induced degree is likely to be lower than k , we will resample v to either use a higher sample rate, or terminate the sample mode.

While the idea is simple, the intricate part is to ensure that at peeling round k , all vertices with coreness k must not be in sample mode and have their the true induced degrees computed. The challenge is then to control the error probability for any vertex to miss its peeling round. Next, we present the details in our sampling algorithm, such that it gives correct answers with high probability.

4.1.1 The Framework with Sampling. We present our framework with sampling in Alg. 4. We first introduce this framework at a high level, and then elaborate on each function in Alg. 5. The framework roughly follows Alg. 1 with a few changes due to sampling. First of all, we start with initializing the sampler of v by SETSAMPLER($v, 0$), which determines whether v should be sampled when $k = 0$, and if so, what its sample rate should be.

Algorithm 5: Functions used in our algorithm with sampling

Parameters: r : when $\tilde{d}[v]$ decrement to a factor of r , we resample v
 $\mu = 4c \ln n$: expected number of hits each sampler, $c > 2$

struct sampler *// For each $v \in V$, maintain a sampler structure $\sigma[v]$*
 $mode$: boolean flag indicating whether v is in sample mode
 $rate$: the sample rate for v
 cnt : the number of hits in the sampling process

1 **Function** PEEL($\mathcal{F}, k$) *// peeling process with sampling*
2 $\mathcal{F}_{next} \leftarrow \emptyset; C \leftarrow \emptyset$ *// C: vertices to recount their induced degrees*
3 **parallel_foreach** $v \in \mathcal{F}$ **do**
4 **parallel_foreach** $u \in N(v)$ **do**
5 **if** $\sigma[u].mode$ **then**
6 $\delta \leftarrow \text{atomic_inc}(\sigma[u].cnt)$ with probability $\sigma[u].rate$
 // If sufficient samples are collected, add u to C
7 **if** $\delta = \mu - 1$ **then** Add u to C
8 **else**
9 $\delta \leftarrow \text{atomic_dec}(\tilde{d}[u])$
10 **if** $\delta = k + 1$ **then** Add u to $\mathcal{F}_{next}$
11 **return** $\langle \mathcal{F}_{next}, C \rangle$

12 **Function** SETSAMPLER(v, k)
 // If the expected induced degree of v is still large and far from k even after decrementing to a factor of r , then v can be sampled safely.
13 **if** $(\tilde{d}[v] \cdot r > k) \wedge (\tilde{d}[v] > \text{threshold})$ **then**
14 $\sigma[v].mode \leftarrow \text{true}$
 // Set the sample rate. This formula is explained in Sec. 4.1.
15 $\sigma[v].rate \leftarrow \mu / ((1 - r) \cdot \tilde{d}[v])$
16 $\sigma[v].cnt \leftarrow 0$
17 **else** $\sigma[v].mode \leftarrow \text{false}$

18 **Function** RESAMPLE($v, k, \mathcal{F}$)
19 $\tilde{d}[v] \leftarrow$ the number of active vertices in $N(v)$
20 **if** $\tilde{d}[v] \leq k$ **then** Add v to $\mathcal{F}$
21 SETSAMPLER(v, k)

22 **Function** VALIDATE(v, k) *// Explained in Sec. 4.1.3*
23 **return** $(\tilde{d}[v] \cdot r > k) \wedge (\sigma[v].cnt < \sigma[v].rate \cdot (\tilde{d}[v] - k) / 4)$

In general, two conditions may trigger updating the sampler of a vertex v . The first case is when k approaches the true induced degree of v , and thus we need to count $\tilde{d}[v]$ more accurately. In particular, when a peeling round starts, we validate all vertices in the sample mode remain safe to be in the sample mode until the next round (lines 5 to 6). If the validation fails, we resample v .

Another case to resample v is when sufficient samples have been collected, and thus we know the induced degree of v has dropped significantly. This happens in the PEEL function. Therefore, PEEL also identifies a set of vertices C that have collected sufficient samples and requires resampling. After the peeling process, we resample all vertices in C (line 10).

In the following, we explain the details of the PEEL function under the sampling scheme, as well as the helper functions SETSAMPLER and RESAMPLE to handle the samplers of each vertex.

4.1.2 Details about the Sampling Scheme. As mentioned, each vertex v maintains a *sampler* structure including three fields: a boolean flag *mode* indicates whether v is in the sample mode, a float *rate* as

the sample rate, and an integer cnt as the number of samples taken by v so far. In general, with sampling rate p and s samples taken, we expect the induced degree to reduce by s/p . However, to enable a high probability guarantee for the estimation s/p , the number of samples need to be $\Omega(\log n)$ (see Sec. 4.1.3). In our algorithm, we use a parameter $\mu = \Theta(\log n)$ to denote the desired number of samples we need to take before we resample v . Namely, we wish to take at least μ samples to ensure high confidence for the estimation of the induced degree for v . When the induced degree of v has decremented by a significant factor, the sample rate has to be adjusted (increased) accordingly. In our case, when the induced degree drops to a factor of r , we resample v . In practice, we use $r = 10\%$. In other words, we use the current sampler to estimate the induced degree of v , until we expect the true $\tilde{d}[v]$ drops to $r \cdot \tilde{d}[v]$, at which point we reset the sampler of v .

We start with presenting our SETSAMPLER function, which initializes the sample parameters for a vertex v . Based on the discussion above, we hope that when μ samples have been taken, we expect the induced degree of v drops from $\tilde{d}[v]$ to $r \cdot \tilde{d}[v]$. Therefore, the sample rate should be set as $\mu / ((1 - r) \cdot \tilde{d}[v])$ (line 15). Accordingly, to determine whether a vertex is safe in sample mode, $r \cdot \tilde{d}[v]$ must still be large enough. In our case, we require it to be larger than a preset threshold, as well as k . The latter condition is because when the expected induced degree approaches k , the probability that the true induced degree is smaller than k will increase dramatically. If $r \cdot \tilde{d}[v]$ is larger than both the sampling threshold and k , we turn on the sample mode for v , and set the parameters accordingly.

We then present our PEEL function with sampling in Alg. 5. This function follows the online peeling process in Alg. 3. The only difference occurs at lines 6 and 7 in Alg. 5, when peeling v and decrementing the induced degree of its neighbor u . If u is in the sample mode, this indicates that u 's current induced degree is far above k . Therefore, instead of decrementing $\tilde{d}[u]$, we increment the number of samples of u , stored in $\sigma[u].cnt$, with probability $\sigma[u].rate$. This increment is also performed atomically, as multiple threads may be accessing u concurrently. If the atomic increment causes $\sigma[u].cnt$ to reach the desired number of samples μ , we add u to C , which buffers all vertices that requires resampling.

To resample a vertex v , we use function RESAMPLE($v, k, \mathcal{F}$). It begins by counting the actual number of active vertices of v , giving the true value of $\tilde{d}[v]$. If $\tilde{d}[v]$ reaches k or lower, the vertex will be added to the current frontier. Then we call SETSAMPLER to reset the parameters for the sampler, based on the new value of $\tilde{d}[v]$.

We note that due to sampling, it is possible that a vertex v is still in the sample mode when the peeling round $k = \kappa[v]$. If so, we are unable to peel v correctly because we do not know its true induced degree at that time. However, we will show that 1) it happens with low probability, and 2) if this happens, we can always detect it, and can restart with stronger sampling parameters. In Sec. 4.1.3, we introduce our approach to handle such potential errors.

4.1.3 Correctness Analysis. We now analyze the correctness of our sampling scheme, and show the error probability is low. The only case for an error is when vertex v is in the sample mode, but the true induced degree of v is less than k at the beginning of round k . We will show that using our VALIDATE() check, the error probability is very low, based on our choices of parameters ($\sigma[v].rate$ and $\sigma[v].cnt$).

LEMMA 4.1. *Assume we toss t coins each with p probability to be a head, and obtain s heads. If $tp \geq 4c \ln n$, then $s \geq tp/4$ whp.*

Proof. This can be shown by using the lower part of the of the multiplicative form of the Chernoff bound. Let s be the number of heads seen. In this case, $\mu = tp$, and $\delta = 1 - s/tp$. Hence we have:

$$\begin{aligned} \Pr\left[s < \frac{tp}{4}\right] &\leq \exp\left(\frac{\delta^2 \mu}{2}\right) = \exp\left(s - \frac{s^2}{2tp} - \frac{tp}{2}\right) \\ &< \exp\left(s - \frac{tp}{2}\right) < \exp\left(-\frac{tp}{4}\right) = n^{-\frac{tp}{4 \ln n}} = n^{-c}. \end{aligned}$$

This proves the lemma. $\square$

With Lem. 4.1, we now prove that Alg. 4 is correct *whp*.

THEOREM 4.2. *For any constant $c \geq 1$, using $\mu = 4(c+2) \ln(n)$, Alg. 4 is correct with probability $1 - n^{-c}$.*

Proof. For a vertex v in sample mode, we use d^* to denote its true induced degree, and $\tilde{d}[v]$ is the induced degree at the point when we last call SETSAMPLER on it (either at the beginning of the algorithm or the last time of resampling). In Lem. 4.1, each increment of sample count on line 6 is a coin toss with probability $p = \sigma[v].rate$. If d^* drops to k or lower, at least $t = \tilde{d}[v] - k$ edges have been removed, corresponding to t coin tosses, and the expected number of sample count is $tp = \sigma[v].rate \cdot (\tilde{d}[v] - k)$.

For a vertex v in the sample mode, we first show that if its induced degree $d^* < k$, then VALIDATE returns *false whp*. First of all, if $k \geq r \cdot \tilde{d}[v]$, the function will fail at the first condition. Otherwise, $k < r \cdot \tilde{d}[v]$. In this case, $tp = (\tilde{d}[v] - k) \cdot \sigma[v].rate$. Plugging in $\sigma[v].rate = \mu / ((1-r) \cdot \tilde{d}[v]) = 4(c+2) \ln n / ((1-r) \cdot \tilde{d}[v])$, we have $tp = 4(c+2) \ln n \cdot ((1-k/\tilde{d}[v]) / (1-r))$. Combining with the assumption that $k < r \cdot \tilde{d}[v]$, we have $tp \geq 4(c+2) \ln n$.

Based on Lem. 4.1, this means that we should have collected $s \geq tp/4 \geq (c+2) \ln n$ heads (successful samples) *whp*. Plugging in $s = \sigma[v].cnt$, $t = \tilde{d}[v] - k$ and $p = \sigma[v].rate$, this means VALIDATE will fail at the second condition. Therefore, if $d^* < k$, VALIDATE will return *false whp*. At this point, we have shown that the error probability for a vertex v to be in sample mode when $d^* < k$ is smaller than $n^{-(c+2)}$ for any constant $c > 0$.

Note that to make the algorithm correct, all the function calls to VALIDATE have to be correct. In Alg. 4, VALIDATE can be called for at most $k_{\max} n \leq n^2$ times. By the union bound, the probability that there exists a failed validation is at most n^{-c} . Therefore, the algorithm is correct with probability at least $1 - n^{-c}$. $\square$

From Thm. 4.2 and the definition of high probability in Sec. 2, we have the following corollary.

COROLLARY 4.3. *Alg. 4 is correct with high probability.*

Note that error may occur at an unsampled vertex, if one of its neighbors is in sample mode and erred. However, such a case must be caused by an error from a sampled vertex. Thus, the algorithm is correct as long as all sampled vertices are processed correctly.

4.1.4 Recover from Errors. Although Thm. 4.2 guarantees that Alg. 4 succeed *whp*, we still need to detect and correct any possible errors caused by sampling, so that Alg. 4 is *Las Vegas* instead of *Monte Carlo*. A possible error happens when a vertex v 's induced degree drops below k in the sample mode before round k . This can be detected when v exits the sampling mode, when we count the true value of $\tilde{d}[v]$. Note that even if this value is smaller than k in round k , it may not be an error since in normal peeling process, when peeling vertices in subrounds, some vertices in k -core may have $\tilde{d}[v]$ drops below k . To verify the correctness, our algorithm will further check whether v 's induced degree is at least k in the previous round. If so, v is still safe and have coreness k . Otherwise, we can restart the algorithm with a larger value of μ or without sampling. However, due to the strong theoretical guarantee Thm. 4.2 provides, we have never encountered restarting in executing our k -core algorithm.

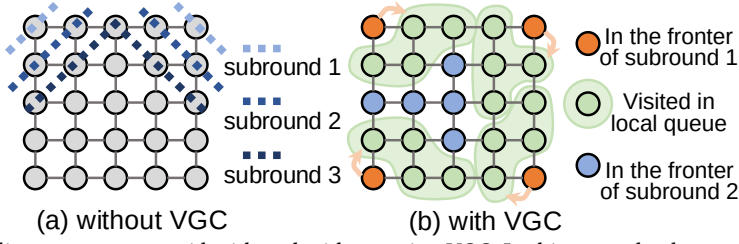


Fig. 3. The peeling process on a grid with and without using VGC. In this example, the queue size is 4. Note that the execution of VGC is not deterministic, and (b) shows a possible execution.

4.1.5 Cost Analysis. Sampling does not affect the work-efficiency of our algorithm. given our setup of our sampling scheme, when a vertex v exits the sampling mode with sufficient samples, $\tilde{d}[v]$ is reduced by a constant fraction *whp*. Therefore, the total cost to recount the true induced degree of v is $O(d(v))$, which add up to at most $O(m)$. The cost validation step is proportional to the number of vertices in the sample mode, and thus is bounded by the active set size. Therefore, the proof in Thm. 3.1 still holds.

Sampling can also reduce contention. As defined in Sec. 2, The contention is the number of possible concurrent operations. If a vertex v is in the sample mode, the concurrent updates will be performed on $\sigma[v].cnt$. For all its $\tilde{d}[v]$ remaining neighbors, each of them will increment $\sigma[v].cnt$ with probability $\sigma[v].rate = \mu / ((1 - r) \cdot \tilde{d}[v])$. Therefore, the contention on $\sigma[v].cnt = \mu / (1 - r) = O((\log n) / (1 - r))$. If a vertex is not in the sample mode (or it has terminated from sample mode), base on the if-condition on line 13, the induced degree is at most $O(k/r + threshold)$. Considering both r and $threshold$ are preset as constants, combining both cases, the contention to handle a specific vertex v is $O(\kappa[v] + \log n)$. For high-degree vertices, this is much smaller than $O(d(v))$, which is the number of concurrent updates to $\tilde{d}[v]$ without sampling.

4.2 Vertical Granularity Control

Our second approach tackles the significant scheduling costs that occur in sparse graphs. In the framework given in Alg. 1, each subround must wait for the completion of the previous one. Between these subrounds, the scheduler distributes tasks to all threads and synchronizes them at the end, which can incur substantial scheduling overhead. This overhead is particularly pronounced in sparse graphs, where vertices typically have low degrees. The computation for processing a low-degree vertex v is small, meaning that the overhead associated with creating and synchronizing the thread for v can outweigh the computational costs. As a result, this overhead diminishes the benefits of parallelism.

Our approach to overcome this challenge is inspired by a recent technique [23, 74] known as *vertical granularity control (VGC)*. Similar to traditional granularity control in parallel programming, VGC focuses on increasing the size of base case tasks in graph processing to mitigate scheduling overhead. In our k -core algorithm, we aim to reduce the overall number of subrounds and eliminate small parallel tasks. To achieve this, we incorporate VGC into our online peeling process in Alg. 3. When peeling a low-degree vertex v , we place all its active neighbors in a FIFO queue, referred to as the *local queue* of v , and process all vertices in the local queue sequentially. When we decrementing the induced degree of a neighbor u , if $\tilde{d}[u]$ drops to k (line 6 in Alg. 3), instead of adding u to $\mathcal{F}_{next}$, we add u to the local queue. This allows u to be processed in the same subround as v , rather than waiting for the next subround. We refer to this process as a *local search* at v .

There are multiple ways to control the granularity in VGC, such as controlling the local queue size, or the number of touched vertices, to be a fixed number. In our code, we simply fix the local

queue size as 128. Once the local queue is full, even if $\tilde{d}[u]$ drops to k , we still add u to the next frontier as normal. An illustration of VGC on a grid is given in Fig. 3. Limiting the work of each local search caps the maximum work each thread receives, thereby ensuring better load balancing. Empirically, processing hundreds of vertices is sufficient to hide scheduling overhead [74]. In our experiments, performance remains relatively stable across queue sizes ranging from hundreds to thousands.

VGC does not change the work-efficiency of the algorithm, as each vertex is still processed exactly once, either from the local queue or from the frontier. In practice, VGC improves the performance for sparse graphs by 1.72–31.2 times.

Some previous work also tried to reduce the number of subrounds. In PKC [38], each processor keeps a subset of $\mathcal{F}$ in a local buffer, and performs a sequential peeling process until the buffer is empty, giving exactly one subround per round. However, it may cause severe load imbalance—if one vertex triggers a chain reaction in peeling, the corresponding processor may perform significantly more work than others. In contrast, our algorithm parallelizes all vertices and use a work-stealing scheduler to enable dynamic load-balancing, and use VGC on top of it to hide scheduling overhead. In summary, VGC improves performance by both hiding synchronization cost between subrounds and achieving good load balancing.

Burdened Span Analysis. To analyze the improvement of VGC, we use the *burdened span* defined in Sec. 2, which charges a factor of ω to each fork/join operation in the span to reflect scheduling overheads. Recall that the burdened span of the framework in Alg. 1 (e.g., the offline algorithm in Julienne) is $\tilde{O}(\omega\rho)$, where ρ is the number of subrounds in the algorithm. Let ρ' be number of subrounds in our algorithm with VGC, and L the total work performed by each local search. In this case, our algorithm with VGC has a span of $\tilde{O}(\rho'(\omega + L))$. As mentioned, we can control L (the granularity of VGC) in various ways, and based on the theory, the primary goal of VGC is to control L asymptotically lower than ω , where ω is on the order of magnitude of 10^4 [37]. Since $\rho' \leq \rho$, the burdened span of our algorithm with VGC is always no worse than the version without VGC. In most of the cases, $\rho' < \rho$, and thus the burdened span can be improved by VGC. In Fig. 7, we empirically showed that the number of subrounds after VGC (i.e., ρ') can be 5–40 times smaller than ρ on the tested graphs. This justifies why our solution with VGC can achieve better parallelism than Julienne.

5 Hierarchical Bucketing Structure

Thm. 3.1 shows that explicitly checking the active vertices in $\mathcal{A}$ (line 9) in each round is asymptotically optimal. However, this simple solution can be slow in practice. To optimize the performance, some existing designs, such as Julienne, choose to use a bucketing structure (defined in Sec. 2) to maintain the active set $\mathcal{A}$. A bucketing structure usually maintains a (partial) mapping from each value d to all vertices in $\mathcal{A}$ with (induced) degree d . In this section, we introduce our new design for the bucketing structure.

We proposed the *hierarchical bucketing structure* (HBS), and show how it improves our k -core algorithm. We note that this data structure may also be of independent interest, since it provides the interface a special parallel priority queue with integer keys, which is useful in many applications [18, 19, 47, 68, 69].

5.1 Interface and Related Work

As mentioned, while refining the active set in each round does not increase the asymptotic cost, it can affect performance due to extra computations and memory accesses. To reduce the overhead, Julienne implements a bucketing structure, which maintains a collection of elements (vertices),

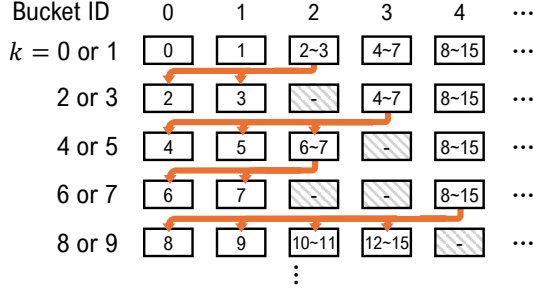


Fig. 4. The execution of the hierarchical bucketing structure for the first 10 rounds of execution.

The number in each box indicates the key (induced degree) range of the associate bucket. The first row shows that a vertex with degree d is initially inserted to bucket $\lceil \log_2(d+1) \rceil$. When $k = 0$ or 1 , vertices with degree 0 or 1 are directly extracted from buckets 0 and 1 . We then redistribute vertices in bucket 2 (with degrees 2 or 3) to buckets 0 and 1 (shown in the second row), such that they can be directly identified when $k = 2$ or 3 . Similarly, after that, we redistributed vertices in bucket 3 (with degrees 4 to 7) to the first three buckets. Vertices with degree 4 and 5 are moved to bucket 0 and 1 , respectively, and vertices with degree 6 and 7 are moved to bucket 2 , so on so forth.

each of which has a unique identifier (vertex ID) and an integer key (induced degree). The structure supports the following three functions (we note that there are more functions in the interface of a bucket structure [18]; For simplicity, we only list the functions used in k -core):

- **BUILDBUCKETS**(R, A): initialize the bucketing structure with key range 0 to R , and insert each element $a \in A$ into it.
- **GETNEXTBUCKET**() $\mapsto \mathcal{F}$: return all elements with the smallest key in the bucketing structure.
- **DECREASEKEY**(a): update a in the structure with its new key.

Julienne maintains a bucketing structure with b buckets. Every b rounds, it generates all frontiers for the next b rounds by **BUILDBUCKETS**($b, \mathcal{A}$), which extract vertices with induced degree $k + i$ to bucket i (k is the current peeling round). Vertices with induced degrees more than $k + 16$ remains in $\mathcal{A}$, which they call a *overflow bucket*. Using an efficient algorithm, the next b frontiers can be generated by one pass of memory access to $\mathcal{A}$, leading to better performance. This strategy reduces the number of accesses to $\mathcal{A}$ by a factor of b . A vertex v will be accessed by **BUILDBUCKETS**() until it is extracted from $\mathcal{A}$, which means $O(\deg(v)/b)$ times of accesses. However, when the induced degree of v is decremented, we also need to move v to the new bucket by **DECREASEKEY**(v). In the worst case, a vertex may be moved $b - 1$ times across the buckets. Therefore, the total cost to process v in the bucketing structure is $O(d(v)/b + b)$, adding up to $O(m/b + nb)$ for all vertices. This function has its minimum value at $b = O(\sqrt{d_{avg}})$, where $d_{avg} = \Theta(m/n)$ is the average degree. Our framework can be viewed as a special case where $b = 1$, which only extracts the next frontier (bucket).

In practice, Julienne uses $b = 16$, which is a reasonable trade-off. Indeed, in our experiments, using $b = 16$ provides better performance than $b = 1$ on most graphs with a large d_{avg} . However, we still observe some challenges with this solution. First, on most graphs with a small d_{avg} , using 16 buckets may cause $20\text{--}70\%$ overhead than using a single bucket. Second, on graphs with very large d_{avg} , the cost of $O(\sqrt{m/n})$ per vertex may still be significant. Next, we propose our new design to overcome these issues.

5.2 Hierarchical Bucketing Structure (HBS)

We first propose the *hierarchical bucketing structure* (HBS) to improve the $O(\sqrt{m/n})$ cost per vertex. Let $d_{\max}$ be the maximum degree in the graph. An HBS maintains $1 + \lceil \log_2(d_{\max} + 1) \rceil$ buckets. Each bucket stores vertices within a range of (induced) degrees of $1, 1, 2, 4, \dots$, growing exponentially. Fig. 4 illustrates an HBS and the (induced) degree range that each bucket maintains in the first ten rounds of execution. When the induced degree of a vertex v drops across a boundary, v is moved to the appropriate new bucket.

To use an HBS in Alg. 1, we will call `BUILDBUCKETS( $d_{\max}, V$ )` at the beginning of the algorithm, use `GETNEXTBUCKET()` to generate the frontier at line 5, and call `DECREASEKEY( $v$ )` when we decrement v 's induced degree. During the execution, we keep a counter k indicating the current minimum key in a HBS, which is also the current round k . Finding the bucket ID of an element is based on the most significant differing bit between its key and k .

Next, we show how these functions are implemented efficiently on HBS. We will maintain each of the $O(\log d_{\max})$ buckets by a parallel hash bag (see Sec. 2) that supports $O(1)$ expected cost for insertion and $O(t)$ work to extract all t elements from the bag. `BUILDBUCKETS()` simply inserts all elements to the corresponding bucket in parallel, based on the most significant bits of their key.

When `GETNEXTBUCKET()` is called, we find the first non-empty bucket with id j . If it is one of the first two lists, we directly call `BAGEXTRACTALL` to return all keys in it. Otherwise, if $j > 2$, we call `BAGEXTRACTALL` to get all elements in this bucket, and redistribute them to the first $j - 1$ buckets, shown as the arrows in Fig. 4.

Finally, when an `DECREASEKEY( $a$ )` is called, we check a 's new bucket ID and insert it to the corresponding bucket. We do not delete a from the original bucket since a hash bag does not support deletion. In this case, an element may have copies in multiple buckets. Hence, when we call `BAGEXTRACTALL`, we also filter out those elements if their keys does not match the bucket ID. We note that this does not increase the asymptotic work of k -core—since a vertex v may be have at most $O(\log d(v))$ copies, the total cost here is at most $\sum \log d(v) = O(m)$.

In our implementation, instead of setting degree ranges for each bucket as $1, 1, 2, 4, 8, \dots$, we set the first eight buckets as single-key bucket. In other words, the first 8 buckets each maintain vertices with degree $k, k+1, \dots, k+7$, and the next buckets correspond to the ranges of $[k+8, k+15]$, $[k+16, k+31]$, and so on. This optimization avoids frequently redistributing vertices in small buckets.

Cost Analysis. The cost of HBS on a vertex v includes inserting v (in `DECREASEKEY()`) or redistributing v (in `GETNEXTBUCKET()`) to a new bucket. In `BUILDBUCKETS()`, v is placed in the $\lceil \log_2(d(v) + 1) \rceil$ -th bucket, and will only move to buckets with smaller IDs. During `DECREASEKEY()` or `GETNEXTBUCKET()`, v can be packed from each bucket for redistribution at most once, and be inserted to any bucket at most once. Hence, the total cost to access v in the bucketing structure is $O(\log d(v))$. Recall that using a fixed number of b buckets leads to $O(d(v)/b + b)$ cost for vertex v , where $d(v)/b + b \geq 2\sqrt{d(v)}$. Therefore, our new design provides a better solution.

5.3 Our Final Design

We now propose our final design of HBS. The cost to handle vertex v in HBS is $O(\log d(v))$, compared to $O(\sqrt{m/n})$ using a fixed number of buckets, or $O(d(v))$ if no bucketing structure is used. We first note that when the average degree is constant, using a bucketing structure offers no benefit, as the overhead may introduce a large hidden constant in the complexity. Therefore, we only use our HBS when the average degree is larger than a constant θ , which is set to 16 in our code. Note that even if the average degree of the original graph is lower than θ , as more low-degree vertices are peeled, the average degree of the graph may become larger. Ideally, when the average

degree surpasses θ , we should switch to HBS. In our code, we simply switch to HBS when a θ -core is reached, which is guaranteed to have an average degree of at least θ .

6 Experiments

	Graph Statistics				Ours			Baselines				Notes	
	Name	n	m	$k_{\max}$	ρ	seq.*	par.	spd.	BZ*	Julienne	ParK		PKC
Social	LJ	4.85M	85.7M	372	3,480	2.37	.203	11.7	1.49	.631	.637	.518	soc-LiveJournal1 [7]
	OK	3.07M	234M	253	5,667	3.94	.526	7.49	3.65	1.23	1.38	.810	com-orkut [79]
	WB	58.7M	523M	193	2,910	29.5	.935	31.6	14.3	1.16	2.64	2.18	soc-sinaweibo [64]
	TW	41.7M	2.41B	2,488	14,964	62.2	2.72	22.9	61.2	4.79	857	75.6	Twitter [44]
	FS	65.6M	3.61B	304	10,034	126	3.68	34.3	174	6.18	416	33.1	Friendster [79]
Geomean for Social Networks						18.5	.999		15.3	1.93	15.3	4.70	
Web	EH	11.3M	522M	9,877	7,393	8.21	.795	10.3	5.49	1.39	5.67	8.22	eu-host [58]
	SD	89.3M	3.88B	10,507	19,063	140	4.39	32.0	143	6.56	410	57.5	sd-arc [58]
	CW	978M	74.7B	4,244	106,819	2453	28.6	85.8	2328	53.2	T/O	T/O	ClueWeb [58]
	HL14	1.72B	124B	4,160	58,737	3587	54.7	65.5	OOM	72.0	OOM	OOM	Hyperlink14 [58]
	HL12	3.56B	226B	10,565	130,737	9177	108	84.6	OOM	152	OOM	OOM	Hyperlink12 [58]
Geomean for Web Networks						622	14.3		N/A	22.1	N/A	N/A	
Road	AF	33.5M	88.9M	3	189	9.83	.155	63.2	5.54	.281	.363	.253	OSM Africa [63]
	NA	87.0M	220M	4	286	32.4	.432	74.8	12.4	.682	.724	.417	OSM North America [63]
	AS	95.7M	244M	4	343	34.8	.480	72.5	16.0	.709	.878	.656	OSM Asia [63]
	EU	131M	333M	4	513	47.4	.679	69.8	33.2	.925	.869	.609	OSM Europe [63]
Geomean for Road Networks						26.9	.385		13.8	.595	.669	.453	
k -NN	CH5	4.21M	29.7M	5	7	.826	.021	39.1	.431	.042	.037	.021	Chem [28, 76], $k = 5$
	GL2	24.9M	65.3M	2	12	6.96	.109	64.1	7.69	.167	.155	.113	GeoLife [76, 86], $k = 2$
	GL5	24.9M	157M	5	42	6.81	.125	54.7	3.54	.196	.179	.249	GeoLife [76, 86], $k = 5$
	GL10	24.9M	310M	10	16	8.46	.162	52.4	5.57	.277	.175	.168	GeoLife [76, 86], $k = 10$
	COS5	321M	1.96B	2	23	117	2.06	56.6	61.9	3.66	2.74	2.08	Cosmo50 [45, 76], $k = 5$
Geomean for k -NN Graphs						8.27	.157		5.26	.268	.218	.183	
Others	TRCE	16.0M	48.0M	2	1,839	2.03	.066	31.0	1.49	1.96	.424	.067	Huge traces [64]
	BBL	21.2M	63.6M	2	1,915	3.18	.077	41.1	3.36	1.80	.203	.081	Huge bubbles [64]
	GRID	100M	400M	2	50,499	6.21	.282	22.1	14.1	14.8	8.03	3.21	Synthetic grid graph
	CUBE	1.00B	6.0B	3	2,895	183	4.01	45.7	162	9.46	110	10.8	Synthetic cubic graph
	HCNS	0.1M	5.0B	50,000	50,000	27.8	2.01	13.8	23.5	16.0	49.7	OOM	Synthetic high-coreness graph
	HPL	100M	1.20B	3,980	6,297	47.3	1.77	26.8	38.9	3.59	30.4	59.1	Synthetic power-law graph
Geomean for Other Graphs						14.6	.523		14.8	5.52	6.97	N/A	

Table 2. Graph information and running time (in seconds) of tested algorithms. $n = |V|$, $m = |E|$. $k_{\max}$ = maximum coreness value. ρ = the peeling complexity [18], i.e., the number of subrounds using offline peeling. “seq.” = our sequential running time, “par.” = our parallel running time. “spd.” = seq./par., i.e., self-relative speedup of our algorithm. Baselines include the BZ algorithm [9], Julienne [18, 19], ParK [17] and PKC [38]. “*” indicates the algorithm is sequential. The best running time on each graph is highlighted in bold. “T/O”: timeout (> 2000 seconds) for parallel algorithms. “OOM”: out of memory.

6.1 Experiment Setup

Our experiments were conducted on a 96-core machine (192 hyper-threads) equipped with four 2.1 GHz Intel Xeon Gold 6252 CPUs, each with a 36MB L3 cache and 1.5TB of main memory. For all tests, except the sequential ones, we used `numactl -i all` to interleave memory across all CPUs. We report the average runtime of five runs, following an initial warm-up run.

6.1.1 Datasets. We tested a wide range of real-world graphs, including large-scale social networks, web graphs, road networks, k -NN graphs, and various other graphs. Additionally, we generate synthetic graphs to simulate adversarial scenarios for both existing baselines and our own algorithm. Directed graphs are symmetrized by converting edges to bidirectional. The graph information, along with their references and acronyms, are shown in Tab. 2, and we will refer to the graphs

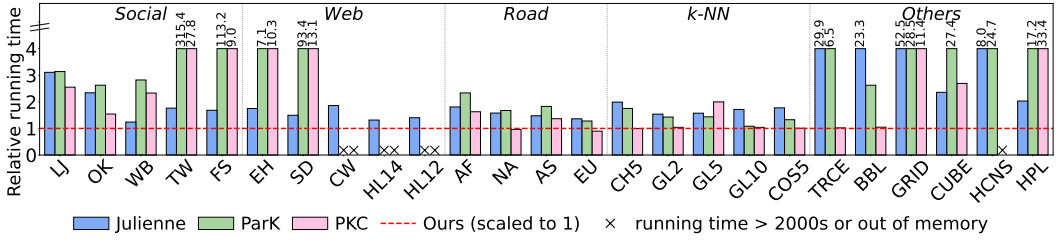


Fig. 5. Relative running time of ParK [17], PKC [38] and Julianne [18, 19] normalized to our running time (red dotted line) on all graphs. Lower is better. The bars are truncated at 4 for better visualization. The text on the bars are actual relative running time.

using these acronyms throughout the rest of this section. The k -NN graphs are generated from a set of vectors (multi-dimensional points). Each vertex represent a point (vector), and it has directed edges to its k nearest neighbors. The five graphs in our experiments are from real-world vector datasets [28, 45, 76, 86]. *TRCE* and *BBL* are meshes taken from individual frames of sequences that resembles two-dimensional adaptive numerical simulations [64]. The two graphs *GRID* and *CUBE* are a $10^4 \times 10^4$ 2D grid and a $10^3 \times 10^3 \times 10^3$ 3D cube, respectively. *HCNS* is a synthetic graph with a high $k_{\max} = 50000$. It contains exactly one vertex with coreness i for $1 \leq i < k_{\max}$, and a dense subgraph with coreness $k_{\max}$. *HPL* is a power-law degree distribution graph, generated using the Barabasi-Albert model [8]. We classify the social networks, web graphs, *HCNS*, and *HPL* as *dense graphs* due to their relatively large average degrees and high coreness, while the remaining graphs are categorized as *sparse graphs*.

6.1.2 Baselines. We compare our algorithm against three state-of-the-art parallel baselines: Julianne [18], ParK [15], PKC [38]. Each algorithm exhibits specific strengths and weaknesses depending on the graph types. These algorithms are discussed in more detail in Sec. 3.2. We also compare our algorithm with the sequential algorithm BZ. The implementation for ParK, PKC, and BZ are from the code of the PKC paper [38]. The implementation for Julianne is from the GBBS library [19], in which Julianne is integrated.

6.2 Experimental Results

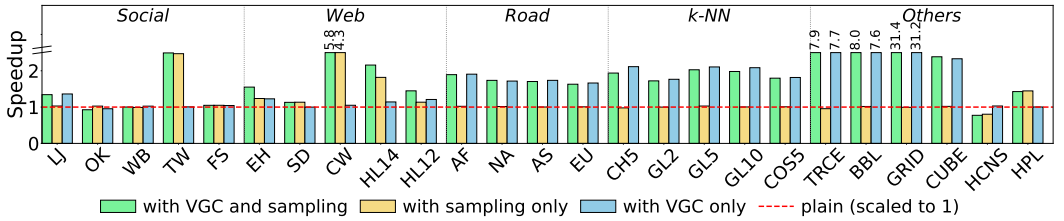


Fig. 6. Speedup of VGC and sampling over a plain implementation. Higher is better. The plain version does not use VGC or sampling. The bars are truncated at 2.5 for better visualization. The text on the bars are the actual speedup values.

6.2.1 Overall Performance. We present the overall performance of our algorithm and all baselines in Tab. 2, and the relative running times of all parallel baselines (normalized to ours) in Fig. 5. In Tab. 2, we highlight the best running time for each graph in bold. We also report the sequential running time of our algorithm along with the self-relative speedup. In most cases, our sequential runtime is comparable to or better than BZ, indicating the work-efficiency of our approach. Our algorithm also demonstrates significant parallelism, achieving a self-relative speedup of 7.5–86×.

In contrast, all baselines exhibit unsatisfactory performance on some graphs due to the lack of parallelism. On certain graphs, they perform even slower than the sequential implementations (BZ or our sequential time) and much slower than other baselines. Our algorithm consistently outperforms the best sequential time by 6.9–85 $\times$. Combining both work-efficiency and strong parallelism, our algorithm is the fastest on 23 out of 25 tested graphs, except for *EU* and *NA*, where it remains competitive and only <12% slower than the best baseline.

Fig. 5 presents relative running time of all parallel implementations, normalized to ours. Each baseline may exhibit tens of times slowdown than our algorithm on certain graphs. Such performance degeneration occurs on different sets of graphs for each baseline, due to the different design of each algorithm.

All algorithms perform well on k -NN graphs, due to their properties: all vertices have small degrees, the same coreness, and only require a few subrounds to complete. Despite small variation in performance, our algorithm is slightly faster on average.

For dense graphs (social networks, web graphs, *HCNS* and *HPL*), which contain many high-degree vertices, Julianne performs well due to work-efficiency and their race-free offline peeling algorithm. PKC and ParK have much worse performance due to 1) work-inefficiency from not maintaining the active set and 2) high contention when updating the induced degrees of the vertices. Our algorithm outperforms Julianne's offline algorithm by 1.24–3.11 $\times$, due to work-efficiency and the sampling scheme that reduces contention during the online peeling process.

On sparse graphs, however, Julianne may have poor performance due to the overhead of enabling race-freedom and fully synchronized subrounds in the offline algorithm. In this case, the simpler online algorithm in PKC and ParK performs better since very light contention is incurred. PKC's thread-local buffer also fully avoids subrounds, and thus it achieves the best performance on three sparse graphs. However, despite being the fastest on three sparse graphs, PKC also leads to the most timeout or out-of-memory cases in our test, and may have much worse performance than others on dense graphs. This highlights the intrinsic difficulty to optimize the performance of parallel k -core algorithms: employing an optimization may address issues on specific graphs, but may sacrifice the performance on other graphs.

The consistent good performance of our algorithm across diverse graph types is enabled by our new techniques. Sampling optimizes for high-degree vertices in dense graphs, while VGC is particularly effective for low-degree vertices in sparse graphs. Consequently, our algorithm consistently performs well on various graph types, and prevents significant performance drops on any type of graphs.

6.2.2 Evaluation on VGC and Sampling. In this section, we evaluate the effectiveness of our online framework using VGC and sampling schemes. We compare four versions of our algorithm: one without sampling or VGC (referred to as the *plain* version), one using only sampling, one using only VGC, and our final with both sampling and VGC. We show the speedup of the latter three over the plain version in Fig. 6. As discussed in Sec. 3, these two techniques are optimized for different scenarios: sampling enhances performance for high-degree vertices, while VGC is designed to optimize for low-degree vertices. In practice, most graphs benefit primarily from one of these techniques. Nearly all sparse graphs show significant improvement with VGC. Seven dense graphs benefit from sampling. On all graphs but *HCNS*, using sampling and/or VGC improves the performance over the plain version.

We first study the impact of sampling. In fact, eight graphs (*TW*, *EH*, *SD*, *CW*, *HL14*, *HL12*, *HPL*, and *HCNS*) contain vertices with very high degrees and trigger sampling. Seven of them (all except *HCNS*) benefit from it. We first discuss the adversarial graph *HCNS* with $k_{\max} = 50000$. In this case, sampling slightly decreases the performance by 24%. The overhead comes from the

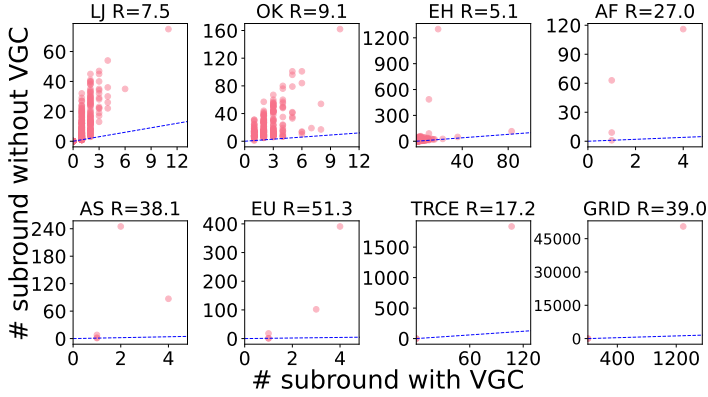


Fig. 7. Number of subrounds with and without VGC. Each point (x, y) means VGC reduces a round with y subrounds to x subrounds. The blue dotted line is the baseline where $y = x$. The number R in each subtitle is the reduction ratio of the number of subrounds.

validation step in each round which processes all vertices in sample mode. On *HCNS*, half of the vertices (all those in $k_{\max}$ -core) need to be checked until $k = \Theta(k_{\max})$. On real-world graphs that trigger sampling (e.g., the power-law graphs), this overhead is minimal since only a small number of vertices have high degrees and enter sampling mode. Therefore, this adversarial graph *HCNS* roughly illustrate the cost of sampling, which is reasonably low (about 24%). For the other seven graphs that trigger sampling, they all benefit from it by up to $4.3\times$ (on *CW*). In fact, while only a small fraction of vertices may require sampling, the benefit is significant. For example, on *TW*, although only about 1000 out of 40 million vertices are in sample mode, the improvement is $2.4\times$. This indicates that a small fraction of high-degree vertices lead to high contention that bottlenecks the entire computation. Our sampling scheme effectively mitigates this contention, resulting in much improved performance.

For VGC, all sparse graphs and certain dense graphs benefit from it, with a speedup of up to $31.2\times$ (on *GRID*). We did not observe notable performance drop due to VGC, which indicates that the overhead of VGC is negligible. To further study the impact of VGC, we compare the number of subrounds with and without VGC on some representative graphs in Fig. 7. For each red point in Fig. 7, the y-coordinate is the number of subrounds without VGC, and the x-coordinate is the number of subrounds using VGC. We can see that the number of subrounds is significantly reduced on various graphs. Even on dense graphs, the numbers of subrounds also decrease by up to $9.1\times$ (on *OK*). However, the improvement in time is not as large as sparse graphs, because the computation on dense graphs is also substantial, and the scheduling overhead does not dominate the time. Therefore, optimizing this part by VGC only marginally improves the performance for dense graphs. On sparse graphs, VGC is very effective. On road networks, the numbers of subrounds in each round are reduced from hundreds to within 4, with an improvement of $26\text{--}51\times$. This also leads to $1.7\text{--}1.9\times$ improvement in time. The improvement is more significant on four other graphs *TRCE*, *BBL*, *GRID*, and *CUBE*, which simulates extreme cases for sparse graphs. The reduce ratio of subrounds ranges from $10\text{--}39\times$, resulting in a speedup of $2.3\text{--}31.2\times$ compared to the plain algorithm.

6.2.3 Evaluation of HBS. We evaluate the effectiveness of HBS by comparing it to two baselines: always using one bucket, as shown in the original framework (Alg. 1), and always using 16 buckets, similar to Julianne’s strategy. Different from the other two technique (VGC and sampling), HBS can be applied to both the online and offline peeling algorithms, and therefore we analyze it separately here. Fig. 8 shows the relative running times (normalized to our method) across

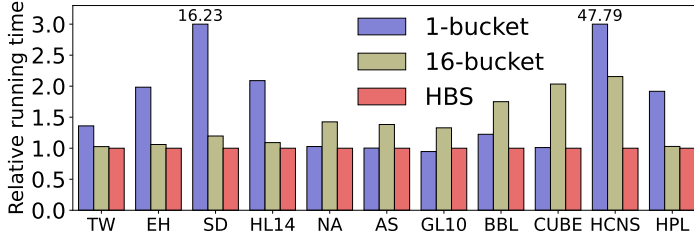


Fig. 8. Relative running time of different bucketing strategies, normalized to HBS. Lower is better.

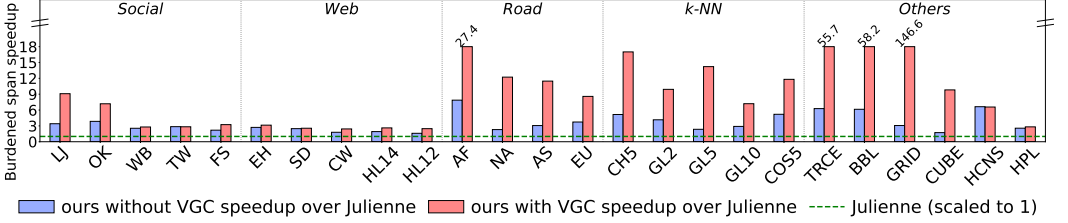


Fig. 9. Burdened span [37] speedup of our algorithm (with and without VGC) over Juliennne [18, 19] (red dotted line, always= 1) on all graphs. Higher is better. The bars are truncated at 18 for better visualization. The text on the bars are actual burdened span speedup.

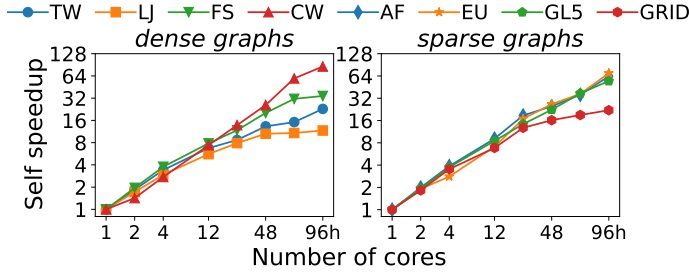


Fig. 10. Self-relative speedup on dense graphs and sparse graphs. “96h” = 96 cores with hyperthreading.

representative graphs. Using one bucket involves constructing the bucket every round, and thus has low performance on dense graphs with large average degrees. Using 16 buckets means to extract and partition vertices into buckets every 16 rounds, which is more efficient on dense graphs. However, as discussed in Sec. 5, on graphs with a constant average degree, using bucket structures does not theoretically improve the cost. The overhead of managing the bucketing structure can make it much slower than using 1 bucket.

Recall that our HBS start with a single bucket for sparse graphs with an average degree of 16 or less, and start to maintain a hierarchy of buckets when k reaches 16, the same threshold. This self-adaptive strategy adjusts to the density of the graph. Across all graphs, HBS matches the performance of the better option between one or 16 buckets, and in some cases, performs much faster than both. For dense graphs, our hierarchical structure is always as good as 16-bucket, and has superior performance on very dense graphs. On the extreme case *HCNS*, using a HBS is 47.8× faster than 1-bucket and 2.01× than 16-bucket structure.

6.2.4 Scalability. The two techniques proposed in Sec. 3 both aim to improve parallelism. To evaluate this, we test the scalability of our algorithm on dense and sparse graphs, respectively, in Fig. 10. The self-relative speedup for all graphs is also reported in Tab. 2. In general, the scalability is more significant on larger graphs, since there are potentially more computation to exploit parallelism. With our new techniques, most sparse graphs achieve a self-relative speedup of 50×

or more. For dense graphs, the two graphs (*CW* and *HL14*) achieves self-speedup exceeding $80\times$. Interestingly, we observe that these two graphs can take advantage from both sampling and VGC (see Fig. 6). This further indicates the effectiveness of our new techniques in improving parallelism.

6.2.5 Burdened Span Analysis. To evaluate how VGC reduces the burdened span and thus improves the performance, we use Cilkview [37] to measure the burdened span of different algorithms in practice. Cilkview only applies to algorithms in OpenCilk [62], and both our code and Julienne can be easily compiled with OpenCilk. We run our algorithm with and without VGC, as well as Julienne [18], on all graphs. Fig. 9 shows the speedup of our algorithm (with and without VGC) over Julienne (higher is better), and the green dotted line at 1 represent the burdened span of Julienne.

As shown in Fig. 9, the burdened span of our plain algorithm (without VGC) is a constant factor ($1.6\text{--}7.9\times$) better than that of Julienne [18], primarily due to the simplicity of the online algorithm. Since Julienne is offline, running histogram and semisort incurs additional rounds of global synchronization. This gap, although as a constant factor in theory, slightly differs the measured burdened span and explains the performance differences.

VGC further reduces the burdened span, particularly on sparse graphs, achieving up to $147\times$ improvement over Julienne. Combining with the results in Fig. 5, the speedup of burdened span matches the speedup of the running time of the algorithms. For example, the three most significant burdened span improvements with VGC are on *TRCE*, *BBL*, and *GRID*, which also show the best time speedups over Julienne. Such highly correlated trends can be observed on other graphs. This suggests that the reduction in burdened span (i.e., reducing the synchronization overhead) is the primary reason for our algorithm to outperform Julienne.

6.2.6 Additional Experiments. In the full paper [52], we also evaluate our algorithm on the task of computing a specific k -core of a graph given a particular value of k , and compare it with a graph library Galois [61]. With different values of k , on two graphs *OK* and *TW*, our algorithm outperforms Galois by $1.6\text{--}6.2\times$.

7 Related Work

The k -core decomposition problem has been extensively studied since the introduction by Seidman [67]. The first sequential algorithm was proposed by Matula and Beck [56], using a bucket sort to arrange vertices by degree and iteratively deleting vertices with degree k (peeling) until all are removed. The algorithm runs in $O(m + n)$. Batagelj and Zaversnik (BZ) [9] provided a sequential implementation with the same time complexity.

The k -core problem is also carefully studied in the shared-memory parallel setting [17, 18, 38, 39, 46, 59], as well as included in many parallel graph libraries such as GraphX [34], Powergraph [33], Ligra [70] and Julienne [18] (later integrated into the GBBS library [19]). In this paper, we compared to the three state-of-the-part solutions, including ParK [17], PKC [38], and Julienne [18]. Among them, ParK and PKC use the online peeling process, and Julienne is offline. More details about them were overviewed in Sec. 3.2.

Given the wide applicability, k -core is also extensively studied in other settings, such as on GPUs [2, 46, 57, 73, 75, 82, 84, 85], the external memory (disk) setting [14, 77], and the low-memory setting [39]. The techniques proposed in these papers mostly focused on the specific challenges in each setting, and have small overlaps with the new techniques introduced in this paper.

We are also aware of variants of k -core decomposition. A direct extension is to maintain k -core with vertex and edge updates. Research has been done on this topic, for both the dynamic (online) setting [3, 5, 29, 49] and the streaming (offline) setting [27, 65, 66]. Another direction is computing approximate k -core decomposition, both in sequential settings [40] and parallel settings [20, 27, 49, 50]. On directed graphs, a similar problem is referred to as D -core decomposition.

Algorithms for it have also been studied recently [32, 48, 53]. The k -core decomposition also relates to many other problems, such as dense subgraph discovery [54] and hierarchical core decomposition [16]. How to apply our new techniques in these related problems can be an interesting future work.

8 Conclusion

In this paper, we present an efficient parallel solution to k -core decomposition. Our contributions include 1) an algorithmic framework that can achieve work-efficiency using various peeling strategies, 2) a sampling scheme to reduce contention on high-degree vertices on dense graphs, 3) a vertical granularity control (VGC) algorithm to hide the scheduling overhead for low-degree vertices on sparse graphs, and 4) a hierarchical bucketing structure (HBS) to improve the performance of managing frontiers. With the new techniques, our algorithm achieves high performance on a variety of graphs. We compare our algorithm with three state-of-the-art algorithm Julienne, Park, and PKC. Due to different design, each baseline may exhibit worse performance than a sequential implementation on certain graphs. Our algorithm consistently performs well on all graphs, and always outperforms the best sequential algorithm by 7.3-84 times. In the experiment, our algorithm is faster than the baselines on all 12 dense graphs and 11 out of 13 sparse graphs. Experimental results also show that each of our new technique is effective in improving performance.

9 Acknowledgement

This work is supported by NSF grants CCF-2103483, TI-2346223, IIS-2227669, NSF CAREER Awards CCF-2238358 and CCF-2339310, the UCR Regents Faculty Development Award, and the Google Research Scholar Program.

References

- [1] Umut A. Acar, Naama Ben-David, and Mike Rainey. 2017. Contention in Structured Concurrency: Provably Efficient Dynamic Non-Zero Indicators for Nested Parallelism. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 75–88.
- [2] Akhlaque Ahmad, Lyuheng Yuan, Da Yan, Guimu Guo, Jieyang Chen, and Chengcui Zhang. 2023. Accelerating k -Core Decomposition by a GPU. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 1818–1831.
- [3] Hidayet Aksu, Mustafa Canim, Yuan-Chi Chang, Ibrahim Korpeoglu, and Özgür Ulusoy. 2014. Distributed k -Core View Materialization and Maintenance for Large Dynamic Graphs. *IEEE Transactions on Knowledge and Data Engineering* 26, 10 (2014), 2439–2452.
- [4] Dan A. Alcantara, Andrei Sharf, Fatemeh Abbasinejad, Shubhabrata Sengupta, Michael Mitzenmacher, John D. Owens, and Nina Amenta. 2009. Real-time parallel hashing on the GPU. *ACM Trans. Graph.* (2009).
- [5] Sabour Aridhi, Martin Brugnara, Alberto Montresor, and Yannis Velegrakis. 2016. Distributed k -core decomposition and maintenance in large dynamic graphs. In *Proceedings of the 10th ACM international conference on distributed and event-based systems*. 161–168.
- [6] Nimar S Arora, Robert D Blumofe, and C Greg Plaxton. 2001. Thread scheduling for multiprogrammed multiprocessors. *Theory of Computing Systems (TOCS)* 34, 2 (2001), 115–144.
- [7] Lars Backstrom, Dan Huttenlocher, Jon Kleinberg, and Xiangyang Lan. 2006. Group formation in large social networks: membership, growth, and evolution. In *ACM International Conference on Knowledge Discovery and Data Mining (SIGKDD)*. 44–54.
- [8] Albert-László Barabási and Réka Albert. 1999. Emergence of scaling in random networks. *science* 286, 5439 (1999), 509–512.
- [9] Vladimir Batagelj and Matjaz Zaversnik. 2003. An $O(m)$ algorithm for cores decomposition of networks. *arXiv preprint cs/0310049* (2003).
- [10] Guy E. Blelloch, Jeremy T. Fineman, Yan Gu, and Yihan Sun. 2020. Optimal parallel algorithms in the binary-forking model. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 89–102.
- [11] Robert D. Blumofe and Charles E. Leiserson. 1998. Space-Efficient Scheduling of Multithreaded Computations. *SIAM J. on Computing* 27, 1 (1998).

- [12] Marián Boguná, Romualdo Pastor-Satorras, Albert Díaz Guilera, and Alex Arenas. 2004. Models of social networks based on social distance attachment. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 70, 5 (2004), 056122.
- [13] Kate Burleson-Lesser, Flaviano Morone, Maria S Tomassone, and Hernán A Makse. 2020. K-core robustness in ecological and financial networks. *Scientific reports* 10, 1 (2020), 3357.
- [14] James Cheng, Yiping Ke, Shumo Chu, and M Tamer Özsu. 2011. Efficient core decomposition in massive networks. In *IEEE International Conference on Data Engineering (ICDE)*. IEEE, 51–62.
- [15] Yizong Cheng, Chen Lu, and Nan Wang. 2013. Local k -core clustering for gene networks. In *2013 IEEE International Conference on Bioinformatics and Biomedicine*. IEEE, 9–15.
- [16] Deming Chu, Fan Zhang, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2022. Hierarchical core decomposition in parallel: From construction to subgraph search. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 1138–1151.
- [17] Naga Shailaja Dasari, Ranjan Desh, and Mohammad Zubair. 2014. ParK: An efficient algorithm for k -core decomposition on multicore processors. In *2014 IEEE International Conference on Big Data (Big Data)*. IEEE, 9–16.
- [18] Laxman Dhulipala, Guy E. Blelloch, and Julian Shun. 2017. Julien: A Framework for Parallel Graph Algorithms using Work-efficient Bucketing. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 293–304.
- [19] Laxman Dhulipala, Guy E. Blelloch, and Julian Shun. 2021. Theoretically efficient parallel graph algorithms can be fast and scalable. *ACM Transactions on Parallel Computing (TOPC)* 8, 1 (2021), 1–70.
- [20] Laxman Dhulipala, Quanquan C Liu, Sofya Raskhodnikova, Jessica Shi, Julian Shun, and Shangdi Yu. 2022. Differential privacy from locally adjustable graph algorithms: k -core decomposition, low out-degree ordering, and densest subgraphs. In *IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE, 754–765.
- [21] Xiangyun Ding, Xiaojun Dong, Yan Gu, Yihan Sun, and Youzhe Liu. 2023. Efficient Parallel Output-Sensitive Edit Distance. In *European Symposium on Algorithms (ESA)*.
- [22] Xiaojun Dong, Laxman Dhulipala, Yan Gu, and Yihan Sun. 2024. Parallel Integer Sort: Theory and Practice. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.
- [23] Xiaojun Dong, Yan Gu, Yihan Sun, and Letong Wang. 2024. Brief Announcement: PASGAL: Parallel And Scalable Graph Algorithm Library. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*.
- [24] Xiaojun Dong, Yan Gu, Yihan Sun, and Yunming Zhang. 2021. Efficient Stepping Algorithms and Implementations for Parallel Shortest Paths. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 184–197.
- [25] Xiaojun Dong, Yunshu Wu, Zhongqi Wang, Laxman Dhulipala, Yan Gu, and Yihan Sun. 2023. High-Performance and Flexible Parallel Algorithms for Semisort and Related Problems. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*.
- [26] Arnold I Emerson, Simeon Andrews, Ikhlaq Ahmed, Thasni KA Azis, and Joel A Malek. 2015. K -core decomposition of a protein domain co-occurrence network reveals lower cancer mutation rates for interior cores. *Journal of clinical bioinformatics* 5 (2015), 1–11.
- [27] Hossein Esfandiari, Silvio Lattanzi, and Vahab Mirrokni. 2018. Parallel and streaming algorithms for k -core decomposition. In *International Conference on Machine Learning (ICML)*. PMLR, 1397–1406.
- [28] Jordi Fonollosa, Sadique Sheik, Ramón Huerta, and Santiago Marco. 2015. Reservoir computing compensates slow response of chemosensor arrays exposed to fast varying gas concentrations in continuous monitoring. *Sensors and Actuators B: Chemical* 215 (2015), 618–629.
- [29] Kasimir Gabert, Ali Pinar, and Ümit V Çatalyürek. 2022. Batch dynamic algorithm to find k -core hierarchies. In *Proceedings of the 5th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*. 1–10.
- [30] Javier García-Algarra, Juan Manuel Pastor, José María Iriondo, and Javier Galeano. 2017. Ranking of critical species to preserve the functionality of mutualistic networks using the k -core decomposition. *PeerJ* 5 (2017), e3321.
- [31] Christos Giatsidis, Dimitrios M Thilikos, and Michalis Vazirgiannis. 2011. Evaluating cooperation in communities with the k -core structure. In *2011 International conference on advances in social networks analysis and mining*. IEEE, 87–93.
- [32] Christos Giatsidis, Dimitrios M Thilikos, and Michalis Vazirgiannis. 2013. D-cores: measuring collaboration of directed graphs based on degeneracy. *Knowledge and information systems* 35, 2 (2013), 311–343.
- [33] Joseph E Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. 2012. PowerGraph: distributed Graph-Parallel computation on natural graphs. In *USENIX conference on Operating Systems Design and Implementation (OSDI)*. 17–30.
- [34] Joseph E Gonzalez, Reynold S Xin, Ankur Dave, Daniel Crankshaw, Michael J Franklin, and Ion Stoica. 2014. GraphX: Graph processing in a distributed dataflow framework. In *USENIX conference on Operating Systems Design and Implementation (OSDI)*. 599–613.
- [35] Yan Gu, Zachary Napier, and Yihan Sun. 2022. Analysis of Work-Stealing and Parallel Cache Complexity. In *SIAM Symposium on Algorithmic Principles of Computer Systems (APOCS)*. SIAM, 46–60.

- [36] Yan Gu, Julian Shun, Yihan Sun, and Guy E. Blelloch. 2015. A Top-Down Parallel Semisort. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 24–34.
- [37] Yuxiong He, Charles E Leiserson, and William M Leiserson. 2010. The Cilkview scalability analyzer. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 145–156.
- [38] Humayun Kabir and Kamesh Madduri. 2017. Parallel k-core decomposition on multicore platforms. In *2017 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*. IEEE, 1482–1491.
- [39] Wissam Khaooud, Marina Barsky, Venkatesh Srinivasan, and Alex Thomo. 2015. K-core decomposition of large networks on a single PC. *Proceedings of the VLDB Endowment* 9, 1 (2015), 13–23.
- [40] Valerie King, Alex Thomo, and Quinton Yong. 2022. Computing $(1 + \epsilon)$ -approximate degeneracy in sublinear time. *arXiv preprint arXiv:2211.04627* (2022).
- [41] Maksim Kitsak, Lazaros K Gallos, Shlomo Havlin, Fredrik Liljeros, Lev Muchnik, H Eugene Stanley, and Hernán A Makse. 2010. Identification of influential spreaders in complex networks. *Nature physics* 6, 11 (2010), 888–893.
- [42] Pranav S Konduri. 2022. *An Implementation of the Parallel K-core Decomposition Algorithm in GraphBLAS*. Ph.D. Dissertation.
- [43] Yi-Xiu Kong, Gui-Yuan Shi, Rui-Jie Wu, and Yi-Cheng Zhang. 2019. k-core: Theories and applications. *Physics Reports* 832 (2019), 1–32.
- [44] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a social network or a news media? In *International World Wide Web Conference (WWW)*. 591–600.
- [45] YongChul Kwon, Dylan Nunley, Jeffrey P Gardner, Magdalena Balazinska, Bill Howe, and Sarah Loebman. 2010. Scalable clustering algorithm for N-body simulations in a shared-nothing cluster. In *International Conference on Scientific and Statistical Database Management*. Springer, 132–150.
- [46] Longlong Li, Hu Chen, Ping Li, Jie Han, Guanghui Wang, and Gong Zhang. 2021. The k-core decomposition algorithm under the framework of GraphBLAS. In *2021 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–7.
- [47] Qingxia Li and Wenhong Wei. 2013. A parallel single-source shortest path algorithm based on bucket structure. In *2013 25th Chinese Control and Decision Conference (CCDC)*. IEEE, 3445–3450.
- [48] Xuankun Liao, Qing Liu, Jiaxin Jiang, Xin Huang, Jianliang Xu, and Byron Choi. 2022. Distributed d-core decomposition over large directed graphs. *arXiv preprint arXiv:2202.05990* (2022).
- [49] Quanquan C Liu, Jessica Shi, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2022. Parallel Batch-Dynamic Algorithms for k-Core Decomposition and Related Graph Problems. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 191–204.
- [50] Quanquan C Liu, Julian Shun, and Igor Zablotchi. 2024. Parallel k-Core Decomposition with Batched Updates and Asynchronous Reads. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 286–300.
- [51] Youzhe Liu, Xiaojun Dong, Yan Gu, and Yihan Sun. 2024. Implementation for parallel k-core decomposition. <https://github.com/ucrparyl/Parallel-KCore>.
- [52] Youzhe Liu, Xiaojun Dong, Yan Gu, and Yihan Sun. 2025. Parallel k-Core Decomposition: Theory and Practice. *arXiv:2502.08042 [cs.DS]* <https://arxiv.org/abs/2502.08042>
- [53] Wensheng Luo, Yixiang Fang, Chunxu Lin, and Yingli Zhou. 2024. Efficient Parallel D-Core Decomposition at Scale. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2654–2667.
- [54] Wensheng Luo, Zhuo Tang, Yixiang Fang, Chenhao Ma, and Xu Zhou. 2023. Scalable algorithms for densest subgraph discovery. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 287–300.
- [55] Fragkiskos D Malliaros, Christos Giatsidis, Apostolos N Papadopoulos, and Michalis Vazirgiannis. 2020. The core decomposition of networks: Theory, algorithms and applications. *The VLDB Journal* 29, 1 (2020), 61–92.
- [56] David W. Matula and Leland L. Beck. 1983. Smallest-last ordering and clustering and graph coloring algorithms. *J. ACM* 30, 3 (1983), 417–427.
- [57] Amir Mehrafra, Sean Chester, and Alex Thomo. 2020. Vectorising k-core decomposition for gpu acceleration. In *Proceedings of the 32nd International Conference on Scientific and Statistical Database Management*. 1–4.
- [58] Robert Meusel, Oliver Lehmberg, Christian Bizer, and Sebastiano Vigna. 2014. Web Data Commons – Hyperlink Graphs. <http://webdatacommons.org/hyperlinkgraph>.
- [59] Alberto Montresor, Francesco De Pellegrini, and Daniele Miorandi. 2011. Distributed k-core decomposition. In *Proceedings of the 30th annual ACM SIGACT-SIGOPS symposium on principles of distributed computing*. 207–208.
- [60] Flaviano Morone, Gino Del Ferraro, and Hernán A Makse. 2019. The k-core as a predictor of structural collapse in mutualistic ecosystems. *Nature physics* 15, 1 (2019), 95–102.
- [61] Donald Nguyen, Andrew Lenharth, and Keshav Pingali. 2014. Deterministic Galois: On-demand, Portable and Parameterless. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [62] OpenCilk 2020. OpenCilk. <https://www.opencilk.org/>.

- [63] OpenStreetMap contributors. 2010. OpenStreetMap. <https://www.openstreetmap.org/>.
- [64] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *AAAI Conference on Artificial Intelligence*. <https://networkrepository.com>
- [65] Ahmet Erdem Sariyüce, Buğra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Ümit V Çatalyürek. 2013. Streaming algorithms for k -core decomposition. *Proceedings of the VLDB Endowment* 6, 6 (2013), 433–444.
- [66] Ahmet Erdem Sariyüce, Buğra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Ümit V Çatalyürek. 2016. Incremental k -core decomposition: algorithms and evaluation. *The VLDB Journal* 25 (2016), 425–447.
- [67] Stephen B. Seidman. 1983. Network structure and minimum degree. *Social networks* 5, 3 (1983), 269–287.
- [68] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2021. Parallel clique counting and peeling algorithms. In *SIAM Conference on Applied and Computational Discrete Algorithms (ACDA)*. SIAM, 135–146.
- [69] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2023. Theoretically and practically efficient parallel nucleus decomposition. In *Proceedings of the 2023 ACM Workshop on Highlights of Parallel Computing*. 7–8.
- [70] Julian Shun and Guy E. Blelloch. 2013. Ligra: A Lightweight Graph Processing Framework for Shared Memory. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 135–146.
- [71] Julian Shun and Guy E Blelloch. 2014. Phase-concurrent hash tables for determinism. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 96–107.
- [72] Shiwen Sun, Xiaoxiao Liu, Li Wang, and Chengyi Xia. 2020. New link attack strategies of complex networks based on k -core decomposition. *IEEE Transactions on Circuits and Systems II: Express Briefs* 67, 12 (2020), 3157–3161.
- [73] Alok Tripathy, Fred Hohman, Duen Horng Chau, and Oded Green. 2018. Scalable k -core decomposition for static graphs using a dynamic graph data structure. In *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 1134–1141.
- [74] Letong Wang, Xiaojun Dong, Yan Gu, and Yihan Sun. 2023. Parallel Strong Connectivity Based on Faster Reachability. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 1, 2 (2023), 1–29.
- [75] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D Owens. 2016. Gunrock: A high-performance graph processing library on the GPU. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 1–12.
- [76] Yiqiu Wang, Shangdi Yu, Laxman Dhulipala, Yan Gu, and Julian Shun. 2021. GeoGraph: A Framework for Graph Processing on Geometric Data. *ACM SIGOPS Operating Systems Review* 55, 1 (2021), 38–46.
- [77] Dong Wen, Lu Qin, Ying Zhang, Xuemin Lin, and Jeffrey Xu Yu. 2018. I/o efficient core graph decomposition: application to degeneracy ordering. *IEEE Transactions on Knowledge and Data Engineering* 31, 1 (2018), 75–90.
- [78] Stefan Wuchty and Eivind Almaas. 2005. Peeling the yeast protein network. *Proteomics* 5, 2 (2005), 444–449.
- [79] Jaewon Yang and Jure Leskovec. 2015. Defining and evaluating network communities based on ground-truth. *Knowledge and Information Systems* 42, 1 (2015), 181–213.
- [80] Hongguang Yao, Huihui Xiao, and Wei Wei. 2022. Study on the Hierarchical Structure of the “Belt and Road” Aviation Network Based on K -Core Analysis. *Discrete Dynamics in Nature and Society* 2022, 1 (2022), 2349523.
- [81] Fan Zhang, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2017. Finding critical users for social network engagement: The collapsed k -core problem. In *AAAI Conference on Artificial Intelligence*, Vol. 31.
- [82] Heng Zhang, Haibo Hou, Libo Zhang, Hongjun Zhang, and Yanjun Wu. 2017. Accelerating core decomposition in large temporal networks using gpus. In *International Conference on Neural Information Processing*. Springer, 893–903.
- [83] Haohua Zhang, Hai Zhao, Wei Cai, Jie Liu, and Wanlei Zhou. 2010. Using the k -core decomposition to analyze the static structure of large-scale software systems. *The Journal of Supercomputing* 53 (2010), 352–369.
- [84] Chen Zhao, Ting Yu, Zhigao Zheng, Yuanyuan Zhu, Song Jin, Bo Du, and Dacheng Tao. 2024. PICO: Accelerating All k -Core Paradigms on GPU. In *Proceedings of the 8th Asia-Pacific Workshop on Networking*. 221–222.
- [85] Chen Zhao, Ting Yu, Zhigao Zheng, Yuanyuan Zhu, Song Jin, Bo Du, and Dacheng Tao. 2024. SpeedCore: Space-efficient and Dependency-aware GPU Parallel Framework for Core Decomposition. In *International Conference on Parallel Processing (ICPP)*. 555–564.
- [86] Yu Zheng, Like Liu, Longhao Wang, and Xing Xie. 2008. Learning transportation mode from raw gps data for geographic applications on the web. In *International World Wide Web Conference (WWW)*. 247–256.

Received October 2024; revised January 2025; accepted February 2025